

دانشگاه صنعتی خواجه نصیرالدین طوسی
دانشکده مهندسی برق

درس یادگیری ماشین

استاد: دکتر امیرحسین نیکوفرد

تمرین سری دوم

گروه جاویدنام سپهر ایران

نام و نام خانوادگی	محمدامین محمدیون شبستری
شماره دانشجویی	۴۰۱۲۲۵۰۳
نام و نام خانوادگی	محمدسبحان سخایی
شماره دانشجویی	۴۰۱۱۹۲۵۳
نام و نام خانوادگی	آرین خوشنویس
شماره دانشجویی	۴۰۱۱۷۹۸۳
تاریخ	اردیبهشت ۱۴۰۵



فهرست مطالب

۶	بخش اول: تحلیل گرادیان و پیاده سازی لایه کانولوشن سفارشی (تحت Pytorch)	۱
	۱.۱ استخراج روابط ریاضی دقیق برای گرادیان های $\frac{\partial L}{\partial M}$ و $\frac{\partial L}{\partial W}$ با فرض ورودی تک کاناله، استراید برابر ۱ و بدون پدینگ	
۶	 (Padding = 0)	
۶	۱.۱.۱ روابط کلی حاکم بر سیستم	
۶	۲.۱.۱ محاسبه گرادیان ماتریس های وزن و ماسک مدل	
۸	۲.۱ پیاده سازی لایه سیستم با استفاده از عملیات Unfold در PyTorch	
۸	۱.۲.۱ ایجاد داده های ورودی و پارامترهای مدل	
۸	۲.۲.۱ پیاده سازی عملیات کانولوشن با استفاده از unfold	
۸	۳.۲.۱ محاسبه تابع خطا و انجام عملیات Backpropagation	
۹	۴.۲.۱ کد	
۱۰	۵.۲.۱ نتایج اجرای برنامه	
۱۰	۶.۲.۱ تحلیل و بررسی نتایج	
۱۱	بخش دوم: معماری Bottleneck در Resnet و مدیریت منابع محاسباتی	۲
۱۱	۱.۲ محاسبه تحلیلی تعداد پارامترهای بلوک Bottleneck در ResNet	
۱۱	۱.۱.۲ فرمول کلی تعداد پارامترها در کانولوشن	
۱۱	۲.۱.۲ محاسبه پارامترهای هر لایه	
۱۲	۳.۱.۲ تعداد کل پارامترهای بلوک	
۱۲	۲.۲ کاهش تعداد پارامترهای بلوک Bottleneck برای اجرای مدل روی دستگاه های لبه	
۱۲	۱.۲.۲ محاسبه تعداد پارامترهای ساختار جدید	
۱۳	۲.۲.۲ تعداد کل پارامترهای بلوک اصلاح شده	
۱۳	۳.۲.۲ تأثیر بر میدان دریافت (Receptive Field)	
۱۳	۴.۲.۲ تأثیر بر ظرفیت استخراج ویژگی	
۱۴	۳.۲ پیاده سازی و بررسی تجربی بلوک Bottleneck در PyTorch	
۱۴	۱.۳.۲ پیاده سازی بلوک در PyTorch	
۱۴	۲.۳.۲ کد	
۱۶	۳.۳.۲ نتایج اجرای برنامه	
۱۶	۴.۳.۲ تحلیل نتایج	
۱۷	بخش سوم: معماری YOLO و ترکیب با معیار Soft Dice Score	۳
۱۷	۱.۳ طراحی تابع خطای YOLO مبتنی بر Soft Dice Score برای بهبود مکان یابی	
۱۷	۱.۱.۳ مقدمه	
۱۷	۲.۱.۳ محدودیت های MSE در مکان یابی	
۱۷	۳.۱.۳ تعریف Soft Dice Score برای Bounding Box	
۱۹	۴.۱.۳ مثال های عددی	



۲۰	فرمول نهایی تابع خطای YOLO-Dice	۵.۱.۳
۲۰	جمع‌بندی	۶.۱.۳
۲۰	پیاده‌سازی در Pytorch و تست کردن یک سناریو	۲.۳
۲۱	کد	۱.۲.۳
۲۷	بررسی ساختار کلاس DiceBoxLoss	۲.۲.۳
۲۸	بررسی ساختار کلاس YOLODiceLoss	۳.۲.۳
۲۹	تحلیل نتایج آزمایش‌ها	۴.۲.۳
۳۰	بصری‌سازی کادرها	۵.۲.۳
۳۱	بخش چهارم: تلفیق ویژگی‌های Denoising و Constrictive Autoencoders	۴
۳۱	تعریف رسمی DAE	۱.۴
۳۱	تحلیل با بسط تیلور برای نویز گاوسی کوچک	۲.۴
۳۲	بسط تیلور درجه اول	۱.۲.۴
۳۲	گرفتن امید ریاضی نسبت به ϵ	۲.۲.۴
۳۳	ارتباط DAE با گرادین چگالی داده	۳.۴
۳۳	شهود هندسی DAE	۴.۴
۳۴	تعریف رسمی CAE	۵.۴
۳۴	تحلیل با بسط تیلور	۶.۴
۳۵	شهود هندسی CAE	۷.۴
۳۵	تقریب DAE به CAE برای نویز گاوسی کوچک	۸.۴
۳۶	جمع‌بندی تفاوت‌ها و شباهت‌های مفهومی	۹.۴
۳۶	نتیجه‌گیری	۱۰.۴
۳۷	توضیح و بررسی سیستم خواسته شده	۱۱.۴
۳۷	پاسخ قسمت اول سوال: تابع هدف جامع DAE+CAE	۱۲.۴
۳۷	فرمول‌بندی پایه	۱.۱۲.۴
۳۹	تقسیم تابع هدف به اجزاء DAE و CAE	۲.۱۲.۴
۳۹	تعبیر هندسی و ارتباط با تشخیص ناهنجاری	۳.۱۲.۴
۴۰	پاسخ قسمت دوم سؤال	۱۳.۴
۴۱	کد	۱.۱۳.۴
۴۶	پاسخ قسمت سوم سؤال و تست کردن	۱۴.۴
۴۶	نتایج آموزش در طول اپیاک‌ها	۱.۱۴.۴
۴۷	نتایج خطای بازسازی برای داده‌های عادی و ناهنجار	۲.۱۴.۴
۴۸	تحلیل نمودار training_losses	۳.۱۴.۴
۴۹	تحلیل نمودار reconstruction_errors	۴.۱۴.۴
۵۰	جمع‌بندی تحلیل‌ها	۵.۱۴.۴



فهرست تصاویر

۳۰	 Prediction Bounding box vs Target Bounding box	۱
۴۹	 Training losses over epochs (Total, Reconstruction, Contractive).	۲
۵۰	 Reconstruction errors for normal samples vs. anomalous sample.	۳



فهرست جداول

۴۷	 Training losses per epoch (Total, Reconstruction, Contractive)	۱
۴۸	 Reconstruction errors for normal and anomalous samples	۲



فهرست برنامه‌ها



۱ بخش اول: تحلیل گرادیان و پیاده سازی لایه کانولوشن سفارشی (تحت Pytorch)

۱.۱ استخراج روابط ریاضی دقیق برای گرادیان‌های $\frac{\partial L}{\partial W}$ و $\frac{\partial L}{\partial M}$ با فرض ورودی تک‌کاناله، استراید برابر ۱ و بدون پدینگ ($Padding = 0$)

۱.۱.۱ روابط کلی حاکم بر سیستم

فرض کنید ورودی سیستم $X \in \mathbb{R}^{H \times W}$ ، ماتریس وزن‌های فیلتر $W \in \mathbb{R}^{K \times K}$ و ماتریس ماسک $M \in \mathbb{R}^{K \times K}$ باشد. فیلتر نهایی که روی ورودی اعمال می‌شود حاصل ضرب درایه‌به‌درایه این دو ماتریس است:

$$W' = W \odot M$$

که در آن $\odot$ نشان‌دهنده ضرب هادامارد (درایه‌به‌درایه) است. با فرض یک ورودی تک‌کاناله، استراید برابر ۱ و بدون پدینگ ($Padding = 0$)، خروجی لایه کانولوشن برابر است با کانولوشن ورودی با فیلتر نهایی:

$$Y = X * W'$$

اگر ابعاد فیلتر $K \times K$ باشد، آنگاه با بسط رابطه کانولوشن در سطح درایه‌ها داریم:

$$y_{i,j} = \sum_{m=0}^{K-1} \sum_{n=0}^{K-1} X_{i+m,j+n} W'_{m,n}$$

با جایگذاری $W' = W \odot M$ خواهیم داشت:

$$y_{i,j} = \sum_{m=0}^{K-1} \sum_{n=0}^{K-1} X_{i+m,j+n} (W_{m,n} M_{m,n})$$

که در آن i, j اندیس‌های موقعیت در خروجی هستند.

تابع خطای مورد استفاده در این مسئله میانگین مربعات خطا (Mean Squared Error) است که به صورت زیر تعریف می‌شود:

$$L = \frac{1}{N} \sum_i \sum_j (Y_{i,j} - \hat{Y}_{i,j})^2$$

که در آن $\hat{Y}$ خروجی هدف (Target) و N تعداد کل عناصر خروجی است.

۲.۱.۱ محاسبه گرادیان ماتریس‌های وزن و ماسک مدل

برای محاسبه گرادیان‌ها از قاعده زنجیره‌ای استفاده می‌کنیم. ابتدا مشتق تابع خطا نسبت به خروجی شبکه محاسبه می‌شود:

$$\frac{\partial L}{\partial Y_{i,j}} = \frac{2}{N} (Y_{i,j} - \hat{Y}_{i,j})$$



اکنون گرادیان نسبت به وزن‌های فیلتر را محاسبه می‌کنیم. با استفاده از قاعده زنجیره‌ای داریم:

$$\frac{\partial L}{\partial W_{m,n}} = \sum_i \sum_j \frac{\partial L}{\partial Y_{i,j}} \frac{\partial Y_{i,j}}{\partial W_{m,n}}$$

با توجه به رابطه تعریف شده برای $y_{i,j}$ داریم:

$$Y_{i,j} = \sum_{m=0}^{K-1} \sum_{n=0}^{K-1} X_{i+m,j+n} (W_{m,n} M_{m,n})$$

بنابراین مشتق آن نسبت به $W_{m,n}$ برابر است با:

$$\frac{\partial Y_{i,j}}{\partial W_{m,n}} = X_{i+m,j+n} M_{m,n}$$

در نتیجه گرادیان وزن‌ها به صورت زیر به دست می‌آید:

$$\frac{\partial L}{\partial W_{m,n}} = \sum_i \sum_j \frac{\partial L}{\partial Y_{i,j}} X_{i+m,j+n} M_{m,n}$$

که می‌توان آن را به صورت زیر بازنویسی کرد:

$$\frac{\partial L}{\partial W_{m,n}} = M_{m,n} \sum_i \sum_j \frac{\partial L}{\partial Y_{i,j}} X_{i+m,j+n}$$

به طور مشابه گرادیان نسبت به ماسک نیز از قاعده زنجیره‌ای به دست می‌آید:

$$\frac{\partial L}{\partial M_{m,n}} = \sum_i \sum_j \frac{\partial L}{\partial Y_{i,j}} \frac{\partial Y_{i,j}}{\partial M_{m,n}}$$

از رابطه خروجی داریم:

$$\frac{\partial Y_{i,j}}{\partial M_{m,n}} = X_{i+m,j+n} W_{m,n}$$

بنابراین:

$$\frac{\partial L}{\partial M_{m,n}} = W_{m,n} \sum_i \sum_j \frac{\partial L}{\partial Y_{i,j}} X_{i+m,j+n}$$

در نهایت می‌توان روابط گرادیان‌ها را به صورت فشرده در قالب عملیات ماتریسی نوشت. اگر

$$\delta = \frac{\partial L}{\partial Y}$$

را گرادیان خطا نسبت به خروجی در نظر بگیریم، آنگاه خواهیم داشت:

$$\frac{\partial L}{\partial W} = M \odot (X \star \delta)$$

$$\frac{\partial L}{\partial M} = W \odot (X \star \delta)$$

که در آن $\star$ نشان‌دهنده عملیات cross-correlation بین ورودی و گرادیان خروجی است. بنابراین گرادیان وزن‌ها و ماسک‌ها هر دو به کانولوشن ورودی با گرادیان خروجی وابسته هستند و تنها تفاوت آن‌ها در ضریب درایه‌ای M و W است.

۲.۱ پیاده‌سازی لایه سیستم با استفاده از عملیات **Unfold** در PyTorch

در این بخش، لایه کانولوشنی که روابط ریاضی آن در قسمت قبل استخراج شد، با استفاده از عملیات **unfold** از کتابخانه‌ی PyTorch پیاده‌سازی می‌شود. در این پیاده‌سازی، از ماژول `nn.Conv2d` استفاده نشده و عملیات کانولوشن به صورت دستی و مبتنی بر ضرب ماتریسی بین پچ‌های ورودی و فیلتر نهایی انجام می‌گیرد.

۱.۲.۱ ایجاد داده‌های ورودی و پارامترهای مدل

به منظور آزمایش عملکرد لایه و بررسی گرادیان‌ها، یک **Tensor** ورودی تصادفی با ابعاد $(1, 1, 5, 5)$ و یک **Tensor** هدف با ابعاد $(1, 1, 3, 3)$ ساخته می‌شود. دو پارامتر قابل آموزش به نام‌های W و M با ابعاد 3×3 برای وزن‌های فیلتر و ماسک تعریف شده‌اند. همان‌گونه که در بخش روابط ریاضی گفته شد، فیلتر نهایی برابر است با ضرب درایه‌به‌درایه دو ماتریس مذکور:

$$W' = W \odot M$$

که در آن $\odot$ نشان‌دهنده ضرب هادامارد یا ضرب درایه‌ای است.

۲.۲.۱ پیاده‌سازی عملیات کانولوشن با استفاده از **unfold**

تابع `torch.nn.functional.unfold` یک عملیات پایه‌ای در PyTorch است که وظیفه‌ی آن استخراج تمام پنجره‌های متوالی (پچ‌ها) از تصویر ورودی است. برای یک ورودی 5×5 و فیلتر با اندازه‌ی هسته‌ی 3×3 ، تابع `unfold` تمامی زیرفرم‌های 3×3 را بدون همپوشانی استخراج کرده و آن‌ها را در قالب یک ماتریس با ابعاد $(9, 9)$ قرار می‌دهد (در این مثال ۹ پچ وجود دارد، هر پچ شامل ۹ درایه است). در ادامه، ماتریس وزن نهایی W' تخت (**Flatten**) شده و در قالب یک بردار ردیفی با طول ۹ قرار می‌گیرد. عملیات کانولوشن معادل ضرب ماتریسی بین این بردار و ماتریس پچ‌ها است و حاصل آن همان خروجی لایه کانولوشنی خواهد بود:

$$output_flat = W'_{flat} \times patches$$

سپس این خروجی دوباره به ابعاد اصلی کانولوشن یعنی $(1, 1, 3, 3)$ تغییر شکل داده می‌شود.

۳.۲.۱ محاسبه تابع خطا و انجام عملیات **Backpropagation**

در گام بعد، تابع خطای میانگین مربعات خطا (**MSE**) بین خروجی شبکه و مقدار هدف محاسبه می‌شود:

$$L = \frac{1}{N} \sum_i \sum_j (Y_{i,j} - \hat{Y}_{i,j})^2$$

با استفاده از تابع `criterion = nn.MSELoss()` در PyTorch، مقدار خطا محاسبه شده و سپس با فراخوانی `loss.backward()` مشتقات یا گرادیان‌های پارامترهای مدل به دست می‌آیند. در فرآیند آموزش، این گرادیان‌ها برای بروزرسانی پارامترهای W و M به کار خواهند رفت.

کد ۴.۲.۱

```
1 import torch
2 import torch.nn as nn
3 import torch.nn.functional as F
4
5 torch.manual_seed(42)
6 input_tensor = torch.randn(1, 1, 5, 5, requires_grad=True) # (Batch, Channel, H, W)
7 target = torch.randn(1, 1, 3, 3)
8 kernel_size = 3
9
10 W = nn.Parameter(torch.randn(kernel_size, kernel_size))
11 M = nn.Parameter(torch.randn(kernel_size, kernel_size))
12
13 W_final = W * M
14
15 patches = F.unfold(input_tensor, kernel_size=(kernel_size, kernel_size))
16 flat_weight = W_final.view(1, -1)
17 output_flat = torch.matmul(flat_weight, patches)
18 output = output_flat.view(1, 1, 3, 3)
19
20 criterion = nn.MSELoss()
21 loss = criterion(output, target)
22 loss.backward()
23
24 print(f"Loss: {loss.item():4f}")
25 print("Gradients for W:")
26 print(W.grad)
27 print("\nGradients for M:")
28 print(M.grad)
```



۵.۲.۱ نتایج اجرای برنامه

پس از اجرای یک مرحله‌ی Forward و سپس Backward، مقدار تابع خطا و گرادیان‌های پارامترهای مدل به صورت نمونه زیر به دست آمده است:

$$Loss = 4.6200$$

$$\nabla_W = \begin{bmatrix} 1.1261 & -1.0485 & 0.1057 \\ 2.9304 & 4.1833 & 0.9999 \\ 0.8946 & -0.9351 & -1.2069 \end{bmatrix}$$

$$\nabla_M = \begin{bmatrix} 0.4340 & -0.3002 & -1.0274 \\ 2.1227 & 2.5464 & -1.0235 \\ -0.9933 & 0.2171 & -6.0293 \end{bmatrix}$$

۶.۲.۱ تحلیل و بررسی نتایج

همان‌گونه که مشاهده می‌شود، مقدار گرادیان‌های حاصل برای پارامترهای W و M همگی مقادیر عددی غیرصفر هستند. این موضوع نشان می‌دهد که مدل به درستی توانسته است جریان گرادیان را از خروجی به پارامترها منتقل کرده و فرآیند backpropagation بدون مشکل اجرا شده است.

از آنجا که تابع خطا MSE نسبت به پارامترهای مدل مشتق‌پذیر است، در هر مرحله‌ی آموزش وزن‌ها و ماسک‌ها با استفاده از این گرادیان‌ها به روز خواهند شد. این نتیجه تأیید می‌کند که پیاده‌سازی لایه کانولوشن با استفاده از عملیات unfold در PyTorch از نظر ریاضی و الگوریتمی سازگار است و رفتار آن با مدل‌های استاندارد Conv2d هم‌خوانی دارد.



۲ بخش دوم: معماری Bottleneck در Resnet و مدیریت منابع محاسباتی

۱.۲ محاسبه تحلیلی تعداد پارامترهای بلوک Bottleneck در ResNet

در معماری ResNet، بلوک‌های Bottleneck برای کاهش بار محاسباتی و در عین حال حفظ توان نمایش مدل به کار می‌روند. ایده اصلی این بلوک‌ها آن است که ابتدا با یک کانولوشن 1×1 تعداد کانال‌ها کاهش یابد، سپس پردازش فضایی با کانولوشن 3×3 روی نمایش فشرده‌تر انجام شود، و در نهایت با یک کانولوشن 1×1 تعداد کانال‌ها دوباره به مقدار اولیه بازگردد. در این مسئله، ورودی بلوک دارای ابعاد زیر است:

$$(Batch = 1, Channels = 256, H = 14, W = 14)$$

ساختار بلوک نیز شامل سه لایه کانولوشنی متوالی به صورت زیر است:

$$1 \times 1 (64), \quad 3 \times 3 (64), \quad 1 \times 1 (256)$$

فرض می‌کنیم در هیچ‌یک از لایه‌ها از Bias و Batch Normalization استفاده نشده است.

۱.۱.۲ فرمول کلی تعداد پارامترها در کانولوشن

تعداد پارامترهای یک لایه کانولوشن دویبعی بدون Bias از رابطه زیر به دست می‌آید:

$$Parameters = K_h \times K_w \times C_{in} \times C_{out}$$

که در آن:

- K_h و K_w به ترتیب ارتفاع و عرض هسته کانولوشن هستند،

- C_{in} تعداد کانال‌های ورودی است،

- C_{out} تعداد فیلترها یا کانال‌های خروجی است.

نکته ابعاد فضایی ورودی یعنی 14×14 و همچنین اندازه بچ برابر با ۱، در تعداد پارامترهای قابل یادگیری تأثیری ندارند و فقط بر حجم محاسبات و ابعاد خروجی لایه‌ها اثر می‌گذارند.

۲.۱.۲ محاسبه پارامترهای هر لایه

لایه اول: کانولوشن 1×1 این لایه برای کاهش تعداد کانال‌ها از ۲۵۶ به ۶۴ استفاده می‌شود. بنابراین:

$$Parameters_1 = 1 \times 1 \times 256 \times 64 = 16384$$



لایه دوم: کانولوشن 3×3 ورودی این لایه خروجی لایه قبل است، بنابراین تعداد کانال‌های ورودی ۶۴ خواهد بود. در نتیجه:

$$\text{Parameters}_2 = 3 \times 3 \times 64 \times 64 = 36864$$

لایه سوم: کانولوشن 1×1 این لایه تعداد کانال‌ها را از ۶۴ به ۲۵۶ باز می‌گرداند. بنابراین:

$$\text{Parameters}_3 = 1 \times 1 \times 64 \times 256 = 16384$$

۳.۱.۲ تعداد کل پارامترهای بلوک

با جمع تعداد پارامترهای سه لایه، تعداد کل پارامترهای قابل یادگیری بلوک Bottleneck برابر خواهد بود با:

$$\text{Parameters Total} = 16384 + 36864 + 16384 = 69632$$

بنابراین، تعداد دقیق پارامترهای این بلوک برابر است با:

$$69632$$

۲.۲ کاهش تعداد پارامترهای بلوک Bottleneck برای اجرای مدل روی دستگاه‌های لبه

در کاربردهایی که مدل باید روی دستگاه‌های لبه (Edge Device) با محدودیت شدید حافظه اجرا شود، کاهش تعداد پارامترهای مدل اهمیت زیادی دارد. یکی از روش‌های رایج برای کاهش پیچیدگی مدل، کاهش تعداد فیلترهای میانی در بلوک Bottleneck است. در اینجا تعداد فیلترهای میانی از 64 به 32 کاهش داده می‌شود. در نتیجه، تعداد پارامترهای هر لایه مجدداً محاسبه می‌شود.

۱.۲.۲ محاسبه تعداد پارامترهای ساختار جدید

لایه اول: کانولوشن 1×1 این لایه تعداد کانال‌ها را از 256 به 32 کاهش می‌دهد. بنابراین تعداد پارامترها برابر است با:

$$1 \times 1 \times 256 \times 32 = 8192$$

لایه دوم: کانولوشن 3×3 در این لایه ورودی و خروجی هر دو دارای 32 کانال هستند. بنابراین:

$$3 \times 3 \times 32 \times 32 = 9216$$

لایه سوم: کانولوشن 1×1 این لایه تعداد کانال‌ها را از 32 به 256 افزایش می‌دهد. بنابراین:

$$1 \times 1 \times 32 \times 256 = 8192$$



۲.۲.۲ تعداد کل پارامترهای بلوک اصلاح شده

با جمع تعداد پارامترهای سه لایه خواهیم داشت:

$$8192 + 9216 + 8192 = 25600$$

در حالت اولیه، تعداد کل پارامترهای بلوک برابر با:

$$69632$$

بود. بنابراین درصد کاهش پارامترها برابر است با:

$$\frac{69632 - 25600}{69632} \times 100$$

$$= 63.24\%$$

در نتیجه با کاهش تعداد فیلترهای میانی از 64 به 32، تعداد پارامترهای این بلوک بیش از 63% کاهش می‌یابد که برای اجرای مدل روی دستگاه‌های لبه با محدودیت حافظه بسیار مفید است.

۳.۲.۲ تأثیر بر میدان دریافت (Receptive Field)

کاهش تعداد فیلترهای میانی هیچ تأثیری بر میدان دریافت ندارد. میدان دریافت تنها به عواملی مانند اندازه فیلترها، مقدار stride بستگی دارد.

از آنجا که اندازه فیلترها 1×1 و 3×3 و مقدار stride در این بلوک تغییری نکرده‌اند، هر نورون در خروجی بلوک همچنان همان ناحیه از تصویر ورودی را مشاهده می‌کند که در حالت اولیه مشاهده می‌کرد.

۴.۲.۲ تأثیر بر ظرفیت استخراج ویژگی

کاهش تعداد فیلترها از 64 به 32 باعث کاهش ظرفیت استخراج ویژگی مدل می‌شود.

کاهش پهنای شبکه با نصف شدن تعداد فیلترها، تعداد نگاشت‌های ویژگی (Feature Maps) نیز کاهش می‌یابد و در نتیجه تعداد الگوهای متفاوتی که لایه 3×3 می‌تواند شناسایی کند کمتر خواهد شد.

ایجاد گلوگاه قوی‌تر کاهش تعداد کانال‌های میانی باعث ایجاد یک Bottleneck شدیدتر در مسیر عبور اطلاعات می‌شود. این موضوع می‌تواند باعث کاهش توان مدل در یادگیری ویژگی‌های پیچیده شود.

در مجموع، این تغییر باعث می‌شود مدل سبک‌تر و سریع‌تر اجرا شود که برای دستگاه‌های لبه بسیار مناسب است، اما ممکن است در برخی مسائل باعث کاهش جزئی در دقت مدل شود، در حالی که میدان دریافت شبکه بدون تغییر باقی می‌ماند.

۳.۲ پیاده‌سازی و بررسی تجربی بلوک Bottleneck در PyTorch

به منظور بررسی درستی محاسبات تحلیلی انجام شده در بخش‌های قبل، دو نسخه از بلوک Bottleneck با استفاده از کتابخانه PyTorch پیاده‌سازی شد. در نسخه اول تعداد فیلترهای میانی برابر با 64 و در نسخه دوم برابر با 32 در نظر گرفته شده است. در هر دو مدل از سه لایه کانولوشنی متوالی زیر استفاده شده است:

$$1 \times 1 \rightarrow 3 \times 3 \rightarrow 1 \times 1$$

در این پیاده‌سازی از ماژول nn.Conv2d استفاده شده و مطابق با فرض مسئله، هیچ Bias‌ای در لایه‌ها در نظر گرفته نشده است.

۱.۳.۲ پیاده‌سازی بلوک در PyTorch

در ابتدا یک کلاس به نام BottleneckBlock تعریف می‌شود که ساختار کامل بلوک را شامل می‌شود. این کلاس سه لایه کانولوشنی را ایجاد کرده و در تابع forward ورودی را به صورت متوالی از این لایه‌ها عبور می‌دهد.

کد ۲.۳.۲

```
1 import torch
2 import torch.nn as nn
3
4 class BottleneckBlock(nn.Module):
5     def __init__(self, in_channels, mid_channels, out_channels):
6         super(BottleneckBlock, self).__init__()
7         # Layer 1: 1x1 conv
8         self.conv1 = nn.Conv2d(in_channels, mid_channels, kernel_size=1, bias=False)
9         # Layer 2: 3x3 conv
10        self.conv2 = nn.Conv2d(mid_channels, mid_channels, kernel_size=3, padding=1, bias=False)
11        # Layer 3: 1x1 conv
12        self.conv3 = nn.Conv2d(mid_channels, out_channels, kernel_size=1, bias=False)
13
14        def forward(self, x):
15            out = self.conv1(x)
16            out = self.conv2(out)
17            out = self.conv3(out)
18            return out
19
20 input_tensor = torch.randn(1, 256, 14, 14)
21
22 model_64 = BottleneckBlock(256, 64, 256)
23 params_64 = sum(p.numel() for p in model_64.parameters())
```

```

۲۴ output_64 = model_64(input_tensor)
۲۵
۲۶ model_32 = BottleneckBlock(256, 32, 256)
۲۷ params_32 = sum(p.numel() for p in model_32.parameters())
۲۸ output_32 = model_32(input_tensor)
۲۹
۳۰ print(f"Model with 64 filters:")
۳۱ print(f"Total Parameters: {params_64}")
۳۲ print(f"Output Shape: {output_64.shape}")
۳۳
۳۴ print(f"\nModel with 32 filters:")
۳۵ print(f"Total Parameters: {params_32}")
۳۶ print(f"Output Shape: {output_32.shape}")
    
```

توضیحات کد در ادامه وظیفه هر بخش از کد توضیح داده شده است:

- فراخوانی کتابخانه‌ها: با استفاده از `torch` و `torch.nn` امکان تعریف شبکه‌های عصبی و لایه‌های کانولوشن فراهم می‌شود.
- تعریف کلاس **BottleneckBlock**: این کلاس از `nn.Module` ارث‌بری کرده و سه لایه کانولوشنی را مطابق ساختار بلوک‌های Bottleneck پیاده‌سازی می‌کند.
- لایه اول: کانولوشن 1×1 برای کاهش تعداد کانال‌ها از `in_channels` به `mid_channels`

$$1 \times 1 \times \text{in_channels} \times \text{mid_channels}$$
- لایه دوم: کانولوشن 3×3 با `padding=1` برای حفظ ابعاد فضایی.

$$3 \times 3 \times \text{mid_channels}^2$$
- لایه سوم: کانولوشن 1×1 برای بازگرداندن تعداد کانال‌ها به `out_channels`.

$$1 \times 1 \times \text{mid_channels} \times \text{out_channels}$$
- تابع **forward**: ورودی به ترتیب از سه لایه عبور می‌کند. هدف تنها بررسی ساختار و محاسبه تعداد پارامترها است، بنابراین تابع فعال‌سازی حذف شده است.
- ورودی تصادفی: تنسوری با ابعاد استاندارد:

$$(1, 256, 14, 14)$$
- ساخت مدل ۶۴ فیلتر: تعداد پارامترها با `numel()` محاسبه شده و خروجی مدل ذخیره می‌شود.
- ساخت مدل ۳۲ فیلتر: مشابه مدل اول اما تعداد فیلترهای میانی نصف شده است، بنابراین انتظار کاهش چشمگیر پارامترها وجود دارد.
- چاپ خروجی: شامل تعداد کل پارامترها و شکل خروجی هر مدل جهت مقایسه با محاسبات تحلیلی.



۳.۳.۲ نتایج اجرای برنامه

پس از اجرای کد، خروجی زیر به دست می‌آید:

```
Model with 64 filters
-----
Total Parameters : 69632
Output Shape      : torch.Size([1, 256, 14, 14])
```

```
Model with 32 filters
-----
Total Parameters : 25600
Output Shape      : torch.Size([1, 256, 14, 14])
```

۴.۳.۲ تحلیل نتایج

تطابق تعداد پارامترها خروجی کد نشان می‌دهد که تعداد پارامترهای محاسبه‌شده دقیقاً با محاسبات نظری بخش‌های قبل مطابقت دارد:

برای مدل با ۶۴ فیلتر 69632

برای مدل با ۳۲ فیلتر 25600

این مطابقت نشان‌دهنده درستی کامل پیاده‌سازی و تحلیل است.

ثابت بودن ابعاد خروجی در هر دو مدل ابعاد خروجی یکسان است:

(1, 256, 14, 14)

این نتیجه به دلیل موارد زیر است:

- تعداد کانال‌های خروجی در هر دو مدل برابر با ۲۵۶ تنظیم شده است.
 - در لایه 3×3 از $\text{padding}=1$ استفاده شده است که باعث حفظ ابعاد فضایی می‌شود.
- در نتیجه، کاهش فیلترهای میانی تنها باعث کاهش تعداد پارامترها می‌شود و هیچ تأثیری بر ابعاد خروجی بلوک ندارد.

۳ بخش سوم: معماری YOLO و ترکیب با معیار Soft Dice Score

۱.۳ طراحی تابع خطای YOLO مبتنی بر Soft Dice Score برای بهبود مکان‌یابی

۱.۱.۳ مقدمه

مدل YOLO یک رگرسور یکپارچه برای تشخیص شیء است که تصویر را به شبکه‌ای $S \times S$ تقسیم کرده و در هر سلول، B جعبه مرزی (Bounding box)، مقدار اعتماد (Confidence) و توزیع احتمال کلاس‌ها را پیش‌بینی می‌کند. بخش مهم این معماری، مکان‌یابی است که تعیین می‌کند مدل چقدر دقیق مختصات و اندازه جعبه را یاد گرفته است. در YOLO کلاسیک این بخش با تابع خطای MSE انجام می‌شود؛ تابعی که به دلیل وابستگی شدید به مقیاس اشیاء، در مواجهه با اشیای کوچک عملکرد ضعیفی دارد. در این نوشتار یک تابع خطای جدید مبتنی بر Soft Dice Score معرفی می‌کنیم که رفتار کاملاً نرمال‌شده داشته و کیفیت همپوشانی هندسی جعبه‌ها را مستقیماً آموزش می‌دهد.

۲.۱.۳ محدودیت‌های MSE در مکان‌یابی

تابع مکان‌یابی YOLO به صورت زیر است:

$$L_{loc}^{MSE} = \lambda_{coord} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathcal{K}_{ij}^{obj} \left[(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 + (\sqrt{w_i} - \sqrt{\hat{w}_i})^2 + (\sqrt{h_i} - \sqrt{\hat{h}_i})^2 \right]$$

مشکل اصلی: MSE برای اشیای کوچک به شدت حساس است، زیرا همان جابه‌جایی چندپیکسلی که برای یک شیء بزرگ ناچیز است، برای شیء کوچک منجر به از دست رفتن کامل همپوشانی می‌شود. اما مدل در هر دو حالت یک جریمه مشابه دریافت می‌کند. این سبب می‌شود YOLO:

- در آموزش روی اشیای کوچک ضعیف‌تر عمل کند،
- گرادین‌های نامرتبط با همپوشانی دریافت کند
- و هماهنگی ضعیفی با معیارهای IoU و همپوشانی واقعی داشته باشد.

۳.۱.۳ تعریف Soft Dice Score برای Bounding Box

معیار Dice برای دو ناحیه A و B :

$$Dice(A, B) = \frac{2|A \cap B|}{|A| + |B|}$$

و نسخه‌ی قابل استفاده در یادگیری:

$$L_{DiceBox} = 1 - \frac{2I + \epsilon}{A_{gt} + A_p + \epsilon}$$

که در آن I مساحت همپوشانی و A_{gt}, A_p مساحت جعبه‌ها هستند. این تابع:



- کاملاً نرمال شده است،
- وابسته به همپوشانی واقعی است
- و نسبت به مقیاس اشیاء پایدار است.

چرا ϵ ضروری است؟ (تحلیل ریاضی و رفتاری) وجود ϵ حیاتی است، چون:

۱. برای جلوگیری از تقسیم بر صفر
اگر در ابتدای آموزش، مدل جعبه‌ای بدون مساحت یا کاملاً اشتباه پیش‌بینی کند، یعنی:

$$A_p = 0, \quad I = 0$$

آن‌گاه مخرج بدون ϵ می‌شود:

$$A_{gt} + A_p = A_{gt}$$

و اگر $A_{gt} = 0$ یا جعبه صفر باشد، مخرج صفر شده و مقدار:

$$\frac{2I}{A_{gt} + A_p}$$

تعریف نشده است. وجود ϵ باعث می‌شود که این حالات مشکلی ایجاد نکنند.

۲. برای پایدار کردن گرادینان هنگام همپوشانی صفر
اگر هیچ همپوشانی وجود نداشته باشد:

$$I = 0$$

آن‌گاه بدون ϵ داریم:

$$L = 1$$

اما گرادینان‌ها بسیار تیز (Sharp) و ناپایدار می‌شوند. اضافه کردن ϵ باعث می‌شود صورت کاملاً صفر نشود:

$$2I + \epsilon = \epsilon$$

و تابع کاملاً شناور و نرم باقی بماند.

۳. برای جلوگیری از جهش ناگهانی Loss هنگام تماس نقطه‌ای

اگر دو جعبه فقط یک تماس کوچک یا ۱ پیکسل همپوشانی داشته باشند:

$$I \approx 1$$

تغییر از $I = 0$ به $I = 1$ باعث یک پرش بسیار بزرگ در مقدار Loss می‌شود. وجود ϵ این پرش را تعدیل و هموار می‌کند.



۴. برای هماهنگی با معیارهای پیوسته در شبکه‌های عصبی

توابع یادگیری مانند Loss Dice باید مشتق‌پذیر و نرم باشند. اما حالت‌های:

$$A_{gt} = 0, \quad A_p = 0, \quad I = 0$$

سه نقطهٔ تکین (Singular) هستند. اضافه کردن ϵ این نقاط را صاف (Smooth) می‌کند.

به طور خلاصه:

برای جلوگیری از ناپایداری عددی و گرادیان لازم است. ϵ

۴.۱.۳ مثال‌های عددی

مثال عددی با تابع خطای Soft Dice score

• مرکز B_{gt} (25, 25)، عرض 8، ارتفاع 6

• مرکز B_p (28, 26)، عرض 8، ارتفاع 6

• مقدار $\epsilon = 10^{-6}$

گام ۱: تبدیل مرکز به گوشه‌ها

$$B_{gt} : (21, 22, 29, 28)$$

$$B_p : (24, 23, 32, 29)$$

گام ۲: محاسبهٔ همپوشانی

$$x_{int1} = \max(24, 21) = 24, \quad x_{int2} = \min(29, 32) = 29, \quad w_{int} = 29 - 24 = 5$$

$$y_{int1} = \max(22, 23) = 23, \quad y_{int2} = \min(28, 29) = 28, \quad h_{int} = 28 - 23 = 5$$

$$I = 5 \times 5 = 25$$

گام ۳: مساحت‌ها

$$A_{gt} = 8 \times 6 = 48, \quad A_p = 8 \times 6 = 48$$

گام ۴: محاسبهٔ Soft Dice Score

$$SoftDiceScore = \frac{2I + \epsilon}{A_{gt} + A_p + \epsilon} = \frac{50 + 10^{-6}}{96 + 10^{-6}}$$

چون مقدار ϵ بسیار کوچک است، مقدار نهایی تفاوت اندکی دارد:

$$Dice \approx 0.520833$$

گام ۵: Dice Loss

$$L_{DiceBox} = 1 - Dice \approx 1 - 0.520833 = 0.479167$$



مثال عددی با تابع خطای MSE در همین مثال، اختلاف مراکز:

$$|x - \hat{x}| = |25 - 28| = 3, \quad |y - \hat{y}| = |26 - 25| = 1$$

اگر صرفاً از MSE استفاده شود:

$$MSE_{xy} = 3^2 + 1^2 = 10$$

نکته مهم:

- با وجود خطای ۳ پیکسلی، جعبه پیش‌بینی هنوز همپوشانی ۵۰٪ دارد و تشخیص کاملاً قابل قبول است.
- ولی MSE این خطا را مانند یک خطای کاملاً بد جریمه می‌کند.
- DiceLoss مستقیماً کیفیت همپوشانی را اندازه‌گیری می‌کند.

۵.۱.۳ فرمول نهایی تابع خطای YOLO-Dice

$$L_{YOLO-Dice} = \sum_{i=0}^{S^2} \sum_{j=0}^B \kappa_{ij}^{obj} L_{DiceBox}(B_{gt}^{(ij)}, B_p^{(ij)}) + \lambda_{noobj} \sum_{i=0}^{S^2} \sum_{j=0}^B \kappa_{ij}^{noobj} (C_i - \hat{C}_i)^2 + \sum_{i=0}^{S^2} \kappa_i^{obj} \sum_c (p_i(c) - \hat{p}_i(c))^2$$

۶.۱.۳ جمع‌بندی

استفاده از DiceBox Loss به جای MSE باعث می‌شود:

- مدل نسبت به اندازه اشیاء نرمال شود،
- همپوشانی هندسی رخ دهد،
- اشیای کوچک بسیار بهتر شناسایی شوند
- و گرادین‌ها معنای هندسی پیدا کنند.

۲.۳ پیاده‌سازی در Pytorch و تست کردن یک سناریو

کد زیر به صورت کامل کلاس‌ها و توابع پیاده شده در پایتورچ را به منظور محاسبه Dice Loss نشان می‌دهد. همچنین در این کد یک سناریوی تست به منظور تعیین عملکرد درست این کلاس‌ها و توابع، پیاده‌سازی شده است. این سناریو همان چیزی است که بخش سوم این سوال از ما می‌خواهد.



```
1
2 # In[2]:
3
4
5 import torch
6 import torch.nn as nn
7
8
9 class DiceBoxLoss(nn.Module):
10     """
11     Computes the Soft Dice Loss for YOLO bounding boxes:
12
13     
$$L = 1 - (2I + \text{eps}) / (A_{\text{gt}} + A_{\text{pred}} + \text{eps})$$

14
15     where boxes are in format (x_center, y_center, w, h).
16     """
17
18     def __init__(self, eps=1e-6):
19         super().__init__()
20         self.eps = eps
21
22     def forward(self, pred_boxes, gt_boxes):
23         """
24         pred_boxes: (batch, S, S, B, 4)
25         gt_boxes:   (batch, S, S, B, 4)
26         """
27
28         # Unpack
29         x_p, y_p, w_p, h_p = pred_boxes[..., 0], pred_boxes[..., 1], pred_boxes[..., 2],
30         ↪ pred_boxes[..., 3]
31
32         x_g, y_g, w_g, h_g = gt_boxes[..., 0], gt_boxes[..., 1], gt_boxes[..., 2],
33         ↪ gt_boxes[..., 3]
34
35
36         # Convert center format ↪ corner format
37
38         x1_p = x_p - w_p / 2
39         y1_p = y_p - h_p / 2
40         x2_p = x_p + w_p / 2
41         y2_p = y_p + h_p / 2
```



```
۳۷
۳۸     x1_g = x_g - w_g / 2
۳۹     y1_g = y_g - h_g / 2
۴۰     x2_g = x_g + w_g / 2
۴۱     y2_g = y_g + h_g / 2
۴۲
۴۳     # Intersection area
۴۴     x1_int = torch.max(x1_p, x1_g)
۴۵     y1_int = torch.max(y1_p, y1_g)
۴۶     x2_int = torch.min(x2_p, x2_g)
۴۷     y2_int = torch.min(y2_p, y2_g)
۴۸
۴۹     w_int = torch.clamp(x2_int - x1_int, min=0)
۵۰     h_int = torch.clamp(y2_int - y1_int, min=0)
۵۱     I = w_int * h_int
۵۲
۵۳     # Areas
۵۴     A_p = w_p * h_p
۵۵     A_g = w_g * h_g
۵۶
۵۷     # Dice box loss
۵۸     dice = (2 * I + self.eps) / (A_g + A_p + self.eps)
۵۹     loss = 1 - dice
۶۰
۶۱     return loss    # shape: (batch, S, S, B)
۶۲
۶۳
۶۴ # In[3]:
۶۵
۶۶
۶۷ class YOLODiceLoss(nn.Module):
۶۸     def __init__(self, S=7, B=2, num_classes=20, lambda_noobj=5.0, eps=1e-6):
۶۹         super().__init__()
۷۰         self.S = S
۷۱         self.B = B
۷۲         self.num_classes = num_classes
۷۳         self.lambda_noobj = lambda_noobj
۷۴
۷۵         self.dice_loss = DiceBoxLoss(eps=eps)
۷۶         self.mse = nn.MSELoss(reduction='none')
```



```
def forward(self, pred, target):  
    """  
    pred, target shapes:  
        (batch, S, S, B*5 + C)  
  
    box format: [x, y, w, h, conf] (per box)  
    class probabilities: C numbers  
    """  
  
    batch = pred.size(0)  
    S, B, C = self.S, self.B, self.num_classes  
  
    # Split predictions  
    pred_boxes = pred[..., :B*5].view(batch, S, S, B, 5)  
    pred_cls = pred[..., B*5:] # (batch, S, S, C)  
  
    # Split targets  
    target_boxes = target[..., :B*5].view(batch, S, S, B, 5)  
    target_cls = target[..., B*5:] # (batch, S, S, C)  
  
    # Object mask  
    obj_mask = target_boxes[..., 4] # shape: (batch, S, S, B)  
    noobj_mask = 1 - obj_mask  
  
    # 1) Localization loss using DiceBox  
    pred_xywh = pred_boxes[..., 0:4]  
    gt_xywh = target_boxes[..., 0:4]  
  
    dice_loss = self.dice_loss(pred_xywh, gt_xywh)  
    dice_loss = dice_loss * obj_mask  
    dice_loss = dice_loss.sum() / batch  
  
    # 2) Confidence loss  
    pred_conf = pred_boxes[..., 4]  
    gt_conf = target_boxes[..., 4]  
  
    conf_loss_obj = self.mse(pred_conf, gt_conf) * obj_mask  
    conf_loss_noobj = self.mse(pred_conf, gt_conf) * noobj_mask
```




```
117         conf_loss = conf_loss_obj.sum() + self.lambda_noobj * conf_loss_noobj.sum()
118         conf_loss /= batch
119
120         # 3) Classification loss
121         # mask per cell (not per box)
122         # cell has object if ANY box has object
123         cell_obj_mask = (obj_mask.sum(dim=3) > 0).float().unsqueeze(-1) # (batch,S,S,1)
124
125         cls_loss = self.mse(pred_cls, target_cls) * cell_obj_mask
126         cls_loss = cls_loss.sum() / batch
127
128         total_loss = dice_loss + conf_loss + cls_loss
129
130         return total_loss, dice_loss, conf_loss, cls_loss
131
132
133     # # Calculating Dice Loss, Classification Loss, Confidence Loss, and Total Loss
134
135     # In[7]:
136
137
138     import torch
139
140     # Test Setting
141     S = 2
142     B = 1
143     C = 1
144     batch = 1
145
146     loss_fn = YOLODiceLoss(S=S, B=B, num_classes=C)
147
148     #####
149     # with overlap
150     #####
151
152     target = torch.zeros((batch, S, S, B*5 + C))
153
154     # Building Target
155     target[0,0,0,0] = 5.0 # x
156     target[0,0,0,1] = 5.0 # y
```



```
157 target[0,0,0,2] = 4.0 # w
158 target[0,0,0,3] = 4.0 # h
159 target[0,0,0,4] = 0.1 # confidence
160 target[0,0,0,5] = 0.1 # class
161
162 # prediction = target
163 pred_perfect = target.clone()
164
165 total, dice, conf, cls = loss_fn(pred_perfect, target)
166
167 print("CASE A: Perfect Prediction")
168 print("Total Loss:", total.item())
169 print("Dice Loss:", dice.item())
170 print("Conf Loss:", conf.item())
171 print("Cls Loss:", cls.item())
172
173
174 #####
175 # With out overlap
176 #####
177
178 pred_bad = target.clone()
179
180 # Moving box
181 pred_bad[0,0,0,0] = 1.0 # x
182 pred_bad[0,0,0,1] = 1.0 # y
183
184 total, dice, conf, cls = loss_fn(pred_bad, target)
185
186 print("\nCASE B: No Overlap")
187 print("Total Loss:", total.item())
188 print("Dice Loss:", dice.item())
189 print("Conf Loss:", conf.item())
190 print("Cls Loss:", cls.item())
191
192
193 # # Visualization
194
195 # In[19]:
196
```



```
۱۹۷
۱۹۸ import matplotlib.pyplot as plt
۱۹۹ import matplotlib.patches as patches
۲۰۰ import numpy as np
۲۰۱
۲۰۲ def draw_boxes(gt, pred):
۲۰۳
۲۰۴     fig, ax = plt.subplots(figsize=(5,5))
۲۰۵
۲۰۶     ax.set_xlim(0,1)
۲۰۷     ax.set_ylim(0,1)
۲۰۸
۲۰۹     # Converting tensor to float
۲۱۰     xg = float(gt[0,0,0,0])
۲۱۱     yg = float(gt[0,0,0,1])
۲۱۲     wg = abs(float(gt[0,0,0,2]))
۲۱۳     hg = abs(float(gt[0,0,0,3]))
۲۱۴
۲۱۵     xp = float(pred[0,0,0,0])
۲۱۶     yp = float(pred[0,0,0,1])
۲۱۷     wp = abs(float(pred[0,0,0,2]))
۲۱۸     hp = abs(float(pred[0,0,0,3]))
۲۱۹
۲۲۰     # center → corner
۲۲۱     xg1 = xg - wg/2
۲۲۲     yg1 = yg - hg/2
۲۲۳     xp1 = xp - wp/2
۲۲۴     yp1 = yp - hp/2
۲۲۵
۲۲۶     # GT box
۲۲۷     gt_box = patches.Rectangle(
۲۲۸         (xg1,yg1),wg,hg,
۲۲۹         linewidth=3,
۲۳۰         edgecolor='green',
۲۳۱         facecolor='green',
۲۳۲         alpha=2.0,
۲۳۳         label="Ground Truth"
۲۳۴     )
۲۳۵
۲۳۶     # Pred box
```

```

۲۳۷     pred_box = patches.Rectangle(
۲۳۸         (xp1,yp1),wp,hp,
۲۳۹         linewidth=3,
۲۴۰         edgecolor='red',
۲۴۱         linestyle='--',
۲۴۲         facecolor='none',
۲۴۳         label="Prediction"
۲۴۴     )
۲۴۵
۲۴۶     ax.add_patch(gt_box)
۲۴۷     ax.add_patch(pred_box)
۲۴۸
۲۴۹     # Plotting the center of the box
۲۵۰     ax.scatter(xg, yg, color='green', s=80)
۲۵۱     ax.scatter(xp, yp, color='red', s=80)
۲۵۲
۲۵۳     ax.legend()
۲۵۴     ax.set_title("Bounding Box Visualization")
۲۵۵     ax.set_aspect('equal')
۲۵۶
۲۵۷     plt.show()
۲۵۸
۲۵۹
۲۶۰     # In[20]:
۲۶۱
۲۶۲
۲۶۳     draw_boxes(target, pred_perfect)
۲۶۴     draw_boxes(target, pred_bad)
۲۶۵
    
```

۲.۲.۳ بررسی ساختار کلاس DiceBoxLoss

تعریف ریاضی تابع زیان دایس در این کلاس، تابع زیان دایس برای ارزیابی دقت کادرهای محدودکننده پیش‌بینی شده نسبت به کادرهای واقعی (Ground Truth) پیاده‌سازی شده است. فرمول استفاده شده برای محاسبه‌ی زیان دایس به صورت زیر است:

$$L = 1 - \frac{2I + \epsilon}{A_{gt} + A_{pred} + \epsilon}$$

که در آن:

- I : مساحت اشتراک (Intersection) بین کادر پیش‌بینی شده و کادر واقعی است.

• A_{gt} : مساحت کادر واقعی.

• A_{pred} : مساحت کادر پیش‌بینی شده.

• ϵ : یک مقدار بسیار کوچک (در اینجا 10^{-6}) برای جلوگیری از خطای تقسیم بر صفر است.

تبدیل مختصات کادرها ورودی‌های این تابع، کادرها را با فرمت (مرکز x ، مرکز y ، عرض، ارتفاع) یا (h, w, y_c, x_c) دریافت می‌کنند. برای محاسبه‌ی مساحت اشتراک، ابتدا باید مختصات به فرمت گوشه‌ها (x_1, y_1, x_2, y_2) تبدیل شوند:

$$\begin{aligned}x_1 &= x_c - \frac{w}{2}, & y_1 &= y_c - \frac{h}{2} \\x_2 &= x_c + \frac{w}{2}, & y_2 &= y_c + \frac{h}{2}\end{aligned}$$

محاسبه‌ی مساحت اشتراک (I) برای یافتن مختصات کادر اشتراک، از عملیات بیشینه و کمینه روی گوشه‌ها استفاده می‌شود:

$$x_{1_{int}} = \max(x_{1_p}, x_{1_g}), \quad y_{1_{int}} = \max(y_{1_p}, y_{1_g})$$

$$x_{2_{int}} = \min(x_{2_p}, x_{2_g}), \quad y_{2_{int}} = \min(y_{2_p}, y_{2_g})$$

عرض و ارتفاع اشتراک با استفاده از تابع `torch.clamp` محاسبه می‌شود تا مقادیر منفی (زمانی که همپوشانی وجود ندارد) صفر شوند:

$$w_{int} = \max(x_{2_{int}} - x_{1_{int}}, 0)$$

$$h_{int} = \max(y_{2_{int}} - y_{1_{int}}, 0)$$

$$I = w_{int} \times h_{int}$$

۳.۲.۳ بررسی ساختار کلاس YOLODiceLoss

پارامترهای اولیه‌ی شبکه این کلاس، تابع زیان کلی YOLO را محاسبه می‌کند. پارامترهای اصلی شامل S (اندازه‌ی گرید)، B (تعداد کادرهای پیش‌بینی شده در هر سلول) و C (تعداد کلاس‌ها) است.

تجزیه‌ی پیش‌بینی‌ها و مقادیر واقعی خروجی شبکه شامل اطلاعات مکان‌یابی کادرها و همچنین احتمالات کلاس‌هاست. در این بخش، تنسورهای پیش‌بینی و واقعی به دو بخش کادرها (مختصات و امتیاز اطمینان) و احتمالات کلاس‌ها تقسیم می‌شوند.

محاسبه‌ی بخش‌های مختلف تابع زیان YOLO ترکیبی از سه بخش است:

۱. زیان مکان‌یابی (**Localization Loss**): این بخش با استفاده از `DiceBoxLoss` محاسبه می‌شود. این زیان تنها برای سلول‌هایی که حاوی شیء هستند (با استفاده از `obj_mask`) در نظر گرفته می‌شود.

۲. زیان اطمینان (**Confidence Loss**): اختلاف بین امتیاز اطمینان پیش‌بینی شده و مقدار واقعی (که ۱ برای وجود شیء و صفر برای عدم وجود آن است) با استفاده از میانگین مربعات خطا (MSE) محاسبه می‌شود. برای سلول‌های بدون شیء، از ضریب λ_{noobj} برای کاهش اثر آن‌ها استفاده می‌شود.

۳. زیان دسته‌بندی (Classification Loss): در صورت وجود شیء در یک سلول، خطای پیش‌بینی کلاس آن نیز با استفاده از MSE محاسبه می‌شود.

در نهایت زیان کل برابر است با مجموع این سه مقدار:

$$L_{total} = L_{dice} + L_{conf} + L_{cls}$$

۴.۲.۳ تحلیل نتایج آزمایش‌ها

در بخش نهایی کد، دو سناریو برای تست این تابع زیان در نظر گرفته شده است.

سناریوی الف: پیش‌بینی بی‌نقص (Perfect Prediction) در این حالت، پیش‌بینی شبکه دقیقاً برابر با مقادیر واقعی است. نتایج خروجی به صورت زیر است:

• زیان کل (Total Loss): 1.788×10^{-7}

• زیان دایس (Dice Loss): 1.788×10^{-7}

• زیان اطمینان (Conf Loss): 0.0

• زیان دسته‌بندی (Cls Loss): 0.0

تحلیل: از آنجا که پیش‌بینی با هدف کاملاً منطبق است، انتظار می‌رود که زیان برابر صفر باشد. با این حال، به دلیل وجود مقدار ϵ در صورت و مخرج کسر محاسبه‌ی دایس ($L = 1 - \frac{2I + \epsilon}{A_g + A_p + \epsilon}$)، مقدار دایس دقیقاً ۱ نمی‌شود و در نتیجه زیان دایس دقیقاً صفر نخواهد بود، بلکه یک مقدار بسیار کوچک (در حدود اسیلون) خواهد داشت. این نشان‌دهنده‌ی صحت عملکرد فرمول است. مقادیر Confidence Loss و Classification Loss برابر با صفر است. زیرا در کد کلا یک کلاس فرض شده است و مقدار اطمینان در تشخیص هم برابر با یک در نظر گرفته شده است. این دو پارامتر فقط به خاطر نشان دادن ساختار کامل YOLO آورده شده است و گرینه هدف محاسبه‌ی DiceLoss و ارزیابی این تابع خطا بوده است.

سناریوی ب: بدون همپوشانی (No Overlap) در این حالت، کادر پیش‌بینی شده به مختصات منتقل می‌شود که هیچ همپوشانی با کادر واقعی ندارد. نتایج خروجی به صورت زیر است:

• زیان کل (Total Loss): 0.9999969

• زیان دایس (Dice Loss): 0.9999969

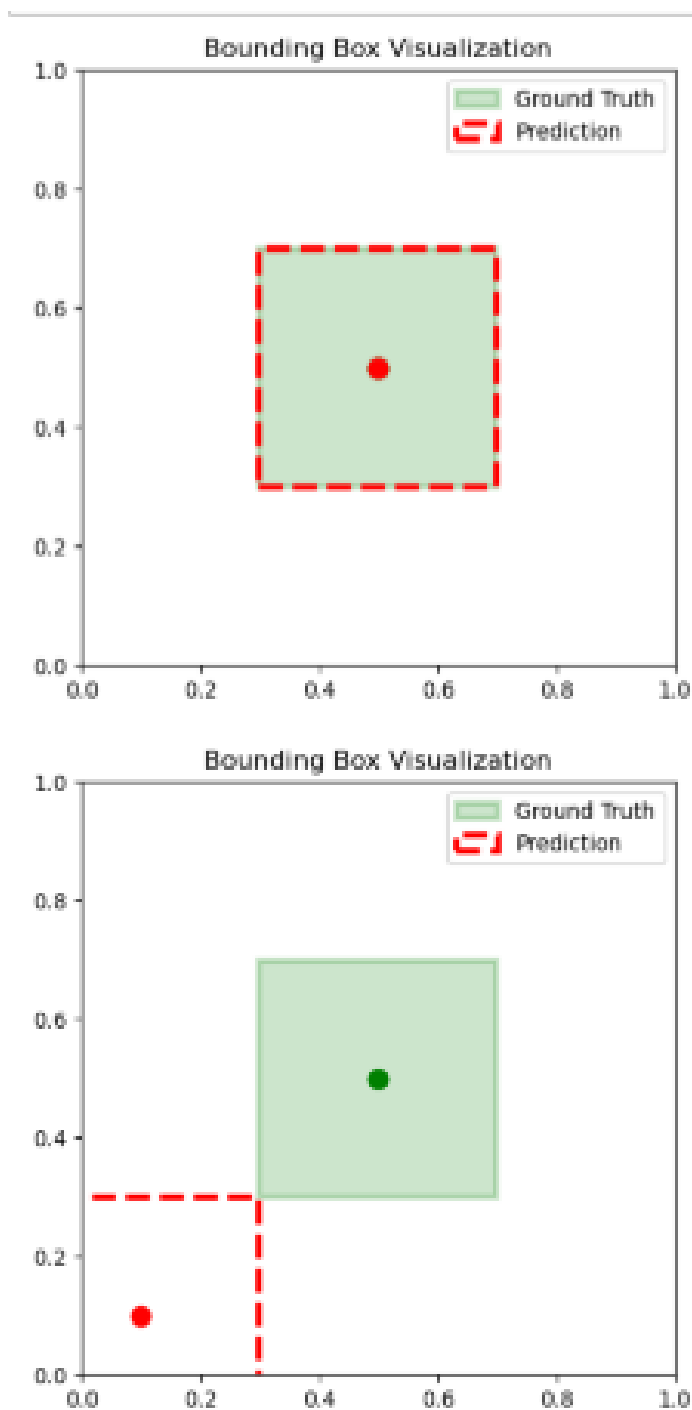
• زیان اطمینان (Conf Loss): 0.0

• زیان دسته‌بندی (Cls Loss): 0.0

تحلیل: در این حالت مساحت اشتراک (I) برابر با صفر است. بنابراین فرمول زیان به $L = 1 - \frac{\epsilon}{A_g + A_p + \epsilon}$ تبدیل می‌شود. این مقدار باید بسیار نزدیک به ۱ باشد. عدد به‌دست‌آمده (0.9999969) به وضوح نشان‌دهنده‌ی درستی منطق عملکرد دایس لاس است. اختلاف بسیار ناچیز آن با عدد ۱ صرفاً به خاطر تأثیر همان مقدار ϵ در صورت کسر است. مقادیر Confidence Loss و Classification Loss برابر با صفر است. زیرا در کد کلا یک کلاس فرض شده است و مقدار اطمینان در تشخیص هم برابر با یک در نظر گرفته شده است. این دو پارامتر فقط به خاطر نشان دادن ساختار کامل YOLO آورده شده است و گرینه هدف محاسبه‌ی DiceLoss و ارزیابی این تابع خطا بوده است.

۵.۲.۳ بصری سازی کادرها

تابع `draw_boxes` نیز با استفاده از کتابخانه `matplotlib` مختصات کادر واقعی و کادر پیش بینی شده را رسم می کند. شکل زیر خروجی مربوط به این بخش است:



شکل ۱: Prediction Bounding box vs Target Bounding box



۴ بخش چهارم: تلفیق ویژگی‌های Denoising و Constrictive Autoencoders

۱.۴ تعریف رسمی DAE

در DAE، ایده اصلی این است که ورودی را عمداً خراب (corrupt) کنیم و از مدل بخواهیم نسخه تمیز آن را بازسازی کند. فرض کنید داده اصلی از توزیع $p_{\text{data}}(x)$ نمونه‌گیری شده است:

$$x \sim p_{\text{data}}(x). \quad (۱)$$

سپس روی ورودی نویز اعمال می‌کنیم. این نویز توسط یک توزیع شرطی $q(\tilde{x} | x)$ تعریف می‌شود:

$$\tilde{x} \sim q(\tilde{x} | x). \quad (۲)$$

نمونه‌ای از نویزهای رایج:

• نویز گاوسی:

$$\tilde{x} = x + \epsilon, \quad \epsilon \sim \mathcal{N}(0, \sigma^2 I), \quad (۳)$$

• نویز ماسک‌گذاری (masking noise) که در آن برخی مختصات x با احتمال مشخصی صفر می‌شوند.

اکنون ورودی شبکه نسخه نویزی $\tilde{x}$ است:

$$h = f_{\theta}(\tilde{x}), \quad \hat{x} = g_{\phi}(h) = g_{\phi}(f_{\theta}(\tilde{x})). \quad (۴)$$

تابع هزینه DAE به صورت زیر تعریف می‌شود:

$$\mathcal{L}_{DAE} = \mathbb{E}_{x \sim p_{\text{data}}} \mathbb{E}_{\tilde{x} \sim q(\tilde{x}|x)} [\ell(x, g_{\phi}(f_{\theta}(\tilde{x})))] \quad (۵)$$

نکته مهم این است که بازسازی باید به x (داده تمیز) نزدیک باشد، نه به $\tilde{x}$ (ورودی نویزی). این باعث می‌شود شبکه یاد بگیرد که چگونه نویز را حذف کند و تنها ساختار پایدار داده را حفظ کند.

۲.۴ تحلیل با بسط تیلور برای نویز گاوسی کوچک

در این بخش می‌خواهیم نشان دهیم که برای نویز گاوسی کوچک، تابع بازسازی DAE چگونه منجر به یک ترم منقبض‌کننده می‌شود و چگونه با گرادینان لگاریتم چگالی داده مرتبط است. فرض کنید نویز گاوسی باشد:

$$\tilde{x} = x + \epsilon, \quad \epsilon \sim \mathcal{N}(0, \sigma^2 I), \quad (۶)$$

و تابع بازسازی را با نماد

$$r(x) := g_{\phi}(f_{\theta}(x)) \quad (۷)$$

نمایش دهیم. آنگاه خروجی مدل برای ورودی نویزی $x + \epsilon$ برابر است با $r(x + \epsilon)$. تابع هزینه DAE برای یک نمونه خاص (با فرض خطای مربعی) به صورت زیر است:

$$\mathcal{L}(x) = \mathbb{E}_{\epsilon} [\|x - r(x + \epsilon)\|^2], \quad (۸)$$

که $\epsilon \sim \mathcal{N}(0, \sigma^2 I)$ است. هدف کمینه‌سازی این تابع نسبت به پارامترهای شبکه است.

۱.۲.۴ بسط تیلور درجه اول

برای ϵ کوچک، می‌توانیم $r(x + \epsilon)$ را حول x بسط تیلور کنیم:

$$r(x + \epsilon) \approx r(x) + J_r(x) \epsilon, \quad (۹)$$

که در آن $J_r(x)$ ژاکوبین تابع r نسبت به ورودی x است:

$$J_r(x) = \frac{\partial r(x)}{\partial x}. \quad (۱۰)$$

اکنون عبارت خطای بازسازی را بسط می‌دهیم:

$$x - r(x + \epsilon) \approx x - (r(x) + J_r(x) \epsilon) = (x - r(x)) - J_r(x) \epsilon. \quad (۱۱)$$

حال مربع نرم را محاسبه می‌کنیم:

$$\|x - r(x + \epsilon)\|^2 \approx \|x - r(x) - J_r(x) \epsilon\|^2 \quad (۱۲)$$

$$= (x - r(x) - J_r(x) \epsilon)^\top (x - r(x) - J_r(x) \epsilon). \quad (۱۳)$$

این عبارت را باز می‌کنیم:

$$\|x - r(x + \epsilon)\|^2 \approx \|x - r(x)\|^2 - 2(x - r(x))^\top J_r(x) \epsilon + \epsilon^\top J_r(x)^\top J_r(x) \epsilon. \quad (۱۴)$$

۲.۲.۴ گرفتن امیدریاضی نسبت به ϵ

اکنون امیدریاضی نسبت به ϵ می‌گیریم. یادآوری می‌کنیم که:

$$\mathbb{E}[\epsilon] = 0, \quad \mathbb{E}[\epsilon \epsilon^\top] = \sigma^2 I. \quad (۱۵)$$

بنابراین:

$$\mathbb{E}[(x - r(x))^\top J_r(x) \epsilon] = (x - r(x))^\top J_r(x) \mathbb{E}[\epsilon] = 0. \quad (۱۶)$$



ترم میانی حذف می‌شود. برای ترم سوم داریم:

$$\mathbb{E}[\epsilon^\top J_r(x)^\top J_r(x) \epsilon] = \mathbb{E}[\text{Tr}(\epsilon^\top J_r(x)^\top J_r(x) \epsilon)] \quad (17)$$

$$= \mathbb{E}[\text{Tr}(J_r(x)^\top J_r(x) \epsilon \epsilon^\top)] \quad (18)$$

$$= \text{Tr}(J_r(x)^\top J_r(x) \mathbb{E}[\epsilon \epsilon^\top]) \quad (19)$$

$$= \text{Tr}(J_r(x)^\top J_r(x) \sigma^2 I) \quad (20)$$

$$= \sigma^2 \text{Tr}(J_r(x)^\top J_r(x)) \quad (21)$$

$$= \sigma^2 \|J_r(x)\|_F^2, \quad (22)$$

که در آن $\|\cdot\|_F$ نورم فروبنیوس است.

در نتیجه:

$$\mathbb{E}_\epsilon[\|x - r(x + \epsilon)\|^2] \approx \|x - r(x)\|^2 + \sigma^2 \|J_r(x)\|_F^2. \quad (23)$$

این رابطه نشان می‌دهد که تابع هزینه DAE، تقریباً برابر است با تابع هزینه یک اتوانکودر معمولی به اضافه یک ترم منظم‌سازی که نورم ژاکوبین بازسازی را جریمه می‌کند. بنابراین برای نویز گاوسی کوچک:

$$\mathcal{L}_{DAE} \approx \mathcal{L}_{AE} + \sigma^2 \mathbb{E}_{x \sim p_{\text{data}}} [\|J_r(x)\|_F^2]. \quad (24)$$

این تقریب، پلی مستقیم بین DAE و CAE ایجاد می‌کند (که در بخش ارتباط دو مدل توضیح داده خواهد شد).

۳.۴ ارتباط DAE با گرادیان چگالی داده

نتیجه بسیار مهمی که توسط (Alain & Bengio (2014) ارائه شده این است که در حالت نویز گاوسی کوچک، نگاشت بازسازی DAE در نقطه x تقریباً به صورت زیر با گرادیان لگاریتم چگالی داده مرتبط است:

$$r(x) - x \approx \sigma^2 \nabla_x \log p_{\text{data}}(x). \quad (25)$$

ایده شهودی این است که اگر DAE برای هر نقطه نویزی $\tilde{x}$ بهترین حدس ممکن از داده تمیز را بدهد، این حدس به طور مؤثری همان میانگین شرطی $E[x_{\text{clean}} | \tilde{x}]$ خواهد بود. می‌توان نشان داد که برای نویز گاوسی کوچک، این میانگین شرطی از رابطه بالا پیروی می‌کند و تفاوت $r(x) - x$ متناسب با گرادیان چگالی است. این نتیجه نشان می‌دهد که DAE در واقع یک denoising vector field را یاد می‌گیرد که نقاط را به نواحی با چگالی بیشتر هدایت می‌کند.

۴.۴ شهود هندسی DAE

فرض کنید داده‌ها روی یک منیفلد کم‌بعد $\mathcal{M}$ در فضای $\mathbb{R}^d$ قرار دارند. نویز گاوسی نقاط را از این منیفلد به اطراف آن پرتاب می‌کند:

$$x \in \mathcal{M}, \quad \tilde{x} = x + \epsilon. \quad (26)$$

مدل DAE یاد می‌گیرد هر نقطه نویزی $\tilde{x}$ را به نزدیک‌ترین نقطه روی منیفلد بازگرداند:

$$r(\tilde{x}) \approx \Pi_{\mathcal{M}}(\tilde{x}), \quad (27)$$

که در آن $\Pi_{\mathcal{M}}$ عملگر پروژکشن (یا نزدیک‌ترین نقطه) روی منیفلد است. بنابراین، DAE یک میدان برداری را یاد می‌گیرد که در تمام نقاط نزدیک به منیفلد، جهت حرکت به سمت انیفلد را نشان می‌دهد. بردار $r(x) - x$ عملاً در راستای گرادیان چگالی داده است و به همین دلیل، DAE نقش مهمی در مدل‌های score-based و diffusion دارد.

۵.۴ تعریف رسمی CAE

اتوانکودر منقبض (Contractive Autoencoder) که توسط Rifai et al. معرفی شده است، به جای آنکه ورودی را خراب کند، مستقیماً حساسیت نمایش نهان نسبت به ورودی را جریمه می‌کند. در CAE داریم:

$$h = f_{\theta}(x), \quad \hat{x} = g_{\phi}(h). \quad (28)$$

تابع هزینه CAE شامل دو بخش است:

$$\mathcal{L}_{CAE} = \mathcal{L}_{\text{reconstruction}}(x, \hat{x}) + \lambda \Omega(f, x), \quad (29)$$

که در آن $\mathcal{L}_{\text{reconstruction}}$ معمولاً خطای بازسازی (مثلاً $\|x - \hat{x}\|^2$) و $\Omega(f, x)$ ترم منقبض‌کننده است که نورم ژاکوبین انکودر را جریمه می‌کند.

ژاکوبین انکودر f نسبت به ورودی x به صورت زیر تعریف می‌شود:

$$J_f(x) = \frac{\partial f(x)}{\partial x}. \quad (30)$$

اگر $f: \mathbb{R}^d \rightarrow \mathbb{R}^k$ باشد، آنگاه $J_f(x)$ یک ماتریس $k \times d$ است که درایه (i, j) آن برابر است با:

$$(J_f(x))_{ij} = \frac{\partial h_i}{\partial x_j}. \quad (31)$$

ترم منقبض‌کننده به صورت نورم فروبنیوس ژاکوبین تعریف می‌شود:

$$\Omega(f, x) = \|J_f(x)\|_F^2 = \sum_{i=1}^k \sum_{j=1}^d \left(\frac{\partial h_i}{\partial x_j} \right)^2. \quad (32)$$

بنابراین تابع هزینه نهایی CAE به صورت زیر است:

$$\mathcal{L}_{CAE} = \mathbb{E}_{x \sim p_{\text{data}}} \left[\|x - g_{\phi}(f_{\theta}(x))\|^2 + \lambda \|J_f(x)\|_F^2 \right]. \quad (33)$$

۶.۴ تحلیل با بسط تیلور

هدف CAE این است که نمایش نهان $h = f(x)$ نسبت به تغییرات کوچک در ورودی x تا حد ممکن پایدار باشد. برای یک نویز کوچک ϵ داریم:

$$x' = x + \epsilon. \quad (34)$$

می‌خواهیم $f(x')$ خیلی با $f(x)$ تفاوت نداشته باشد:

$$f(x + \epsilon) \approx f(x). \quad (35)$$

با بسط تیلور درجه اول:

$$f(x + \epsilon) \approx f(x) + J_f(x)\epsilon. \quad (36)$$

بنابراین تغییر نمایش نهان برابر است با:

$$\Delta h = f(x + \epsilon) - f(x) \approx J_f(x)\epsilon. \quad (37)$$

نرم مربع این تغییر:

$$\|\Delta h\|^2 \approx \epsilon^\top J_f(x)^\top J_f(x) \epsilon. \quad (38)$$

اگر ϵ یک نویز با کواریانس کوچک (مثلاً گاوسی با $\mathbb{E}[\epsilon\epsilon^\top] = \sigma^2 I$) باشد، امیدریاضی این مقدار تقریباً برابر است با:

$$\mathbb{E}[\|\Delta h\|^2] \approx \mathbb{E}[\epsilon^\top J_f(x)^\top J_f(x) \epsilon] \quad (39)$$

$$= \text{Tr}(J_f(x)^\top J_f(x) \mathbb{E}[\epsilon\epsilon^\top]) \quad (40)$$

$$= \sigma^2 \text{Tr}(J_f(x)^\top J_f(x)) \quad (41)$$

$$= \sigma^2 \|J_f(x)\|_F^2. \quad (42)$$

در نتیجه، اگر بخواهیم $\|\Delta h\|^2$ کوچک باشد (یعنی $f(x)$ نسبت به تغییرات کوچک x پایدار باشد)، باید $\|J_f(x)\|_F^2$ کوچک شود. دقیقاً به همین دلیل است که CAE نورم فروبنیوس ژاکوبین را جریمه می‌کند.

۷.۴ شهود هندسی CAE

فرض کنید داده‌ها روی یک منیفلد کم بعد $\mathcal{M}$ قرار دارند. در فضای ورودی، دو نوع جهت اصلی داریم:

- جهت‌های مماسی به منیفلد (tangent directions) که تغییرات واقعی داده را توصیف می‌کنند؛
- جهت‌های عمود بر منیفلد (normal directions) که تغییرات «غیرواقعی» یا نویزی هستند.

هدف CAE این است که:

- در جهت‌های مماسی، مدل باید تا حدی حساس باشد تا بتواند تغییرات واقعی داده را کد کند؛
- در جهت‌های عمود، مدل باید حساسیت بسیار کمی داشته باشد تا نویز را نادیده بگیرد.

جریمه کردن ژاکوبین انکودر باعث می‌شود نگاشت f در اطراف منیفلد منقبض شود، یعنی نقاط نزدیک به یکدیگر در ورودی، نگاشت‌های بسیار نزدیک به هم در فضای نهان داشته باشند، به خصوص در جهت‌های عمود بر منیفلد. این رفتار باعث می‌شود CAE یک نمایش نهان پایدار و «فشرده» از ساختار داده بیاموزد.

۸.۴ تقریب DAE به CAE برای نویز گاوسی کوچک

در بخش تحلیل DAE نشان دادیم که برای نویز گاوسی کوچک، تابع هزینه DAE تقریباً به صورت زیر است:

$$\mathcal{L}_{DAE} \approx \mathcal{L}_{AE} + \sigma^2 \mathbb{E}_x [\|J_r(x)\|_F^2], \quad (43)$$



که در آن $r(x) = g_\phi(f_\theta(x))$ تابع بازسازی است.
در CAE (معادله (۳۳)) تابع هزینه به صورت زیر است:

$$\mathcal{L}_{CAE} = \mathbb{E}_x [\|x - g_\phi(f_\theta(x))\|^2 + \lambda \|J_f(x)\|_F^2]. \quad (44)$$

اگر فرض کنیم که بخش اصلی انقباض توسط انکودر اعمال می شود و ژاکوبین بازسازی $J_r(x)$ و ژاکوبین انکودر $J_f(x)$ به صورت نزدیکی مرتبط یا متناسب هستند (این فرض در بسیاری از تنظیمات عملی و تحلیل های نظری پذیرفته می شود)، می توان نوشت:

$$\|J_r(x)\|_F^2 \approx C \|J_f(x)\|_F^2 \quad (45)$$

برای یک ثابت مثبت C (وابسته به ساختار دیکودر). در این صورت، ترم منظم سازی DAE و CAE از نظر فرم کاملاً مشابه می شوند؛ تنها تفاوت در ضریب وزن دهی است:

$$\sigma^2 C \text{ در DAE در مقابل } \lambda \text{ در CAE.} \quad (46)$$

بنابراین، برای نویز گاوسی کوچک، DAE را می توان به صورت یک CAE با ضریب منظم سازی خاص تعبیر کرد. به بیان دیگر:

$$DAE \approx CAE \quad \text{برای نویز گاوسی کوچک (یعنی } \sigma^2 \text{ کوچک).} \quad (47)$$

این رابطه نشان می دهد که افزودن نویز در DAE به طور ضمنی رفتار منقبض کننده CAE را اعمال می کند.

۹.۴ جمع بندی تفاوت ها و شباهت های مفهومی

- هر دو مدل (CAE و DAE) تلاش می کنند ساختاری پایدار و کم بعد از داده ها یاد بگیرند و نقاط را در اطراف منیفولد داده منقبض کنند.
- DAE این کار را از طریق افزودن نویز به ورودی و مجبور کردن مدل به بازسازی داده تمیز انجام می دهد. تحلیل ریاضی نشان می دهد این روند به طور ضمنی یک ترم منظم سازی بر نورم ژاکوبین ایجاد می کند.
- CAE همان اثر را به صورت صریح و مستقیم اعمال می کند: نورم ژاکوبین انکودر در تابع هزینه ظاهر می شود و انقباض موضعی را تضمین می کند.
- از دید هندسی:

- DAE یاد می گیرد که نقاط نویزی را به منیفولد داده پروژکت کند؛

- CAE یاد می گیرد که نگاشت نهان در جهت های عمود بر منیفولد تقریباً ثابت باشد (انقباض شدید)، در حالی که در جهت های مماسی حساسیت کافی داشته باشد.

۱۰.۴ نتیجه گیری

در این پاسخ نامه نشان دادیم که:



۱. DAE با افزودن نویز گاوسی کوچک، تابع بازسازی ای را یاد می‌گیرد که علاوه بر بازسازی داده تمیز، یک ترم منطبق با نورم ژاکوبین دارد و حتی می‌توان نشان داد که $r(x) - x$ متناسب با $\nabla_x \log p_{\text{data}}(x)$ است.

۲. CAE به صورت مستقیم ژاکوبین انکودر را جریمه می‌کند و بنابراین نگاشت نهان را نسبت به تغییرات کوچک ورودی، به‌ویژه در جهت‌های عمود بر منیفلد، منقبض می‌کند.

۳. برای نویز گاوسی کوچک، تابع هزینه DAE را می‌توان به صورت تقریب خوبی از تابع هزینه CAE همراه با یک ضریب منظم‌سازی خاص دید؛ بنابراین DAE و CAE از نظر رفتار کلی بسیار به یکدیگر نزدیک‌اند.

این تحلیل نشان می‌دهد که هر دو مدل از دیدگاه یادگیری منیفلد، بازسازی نویز و انقباض موضعی نقش مشابهی دارند و می‌توان آن‌ها را دو پیاده‌سازی مختلف یک ایده مشترک دانست: یادگیری ساختار کم‌بعد، پایدار و مقاوم به نویز داده‌ها.

۱۱.۴ توضیح و بررسی سیستم خواسته شده

هدف ما طراحی یک سامانه تشخیص ناهنجاری برای سیگنال‌های زمانی حساس است که از یک encoder-decoder عصبی استفاده می‌کند و هم‌زمان دارای دو ویژگی کلیدی است:

۱. مقاومت در برابر نویز ورودی (Denoising): یعنی اگر سیگنال ورودی دچار اغتشاشات کوچک و تصادفی شود، نمایش پنهان (latent representation) و خروجی بازسازی‌شده تا حد امکان تغییر نکنند.

۲. کنترل مشتقات فضای پنهان نسبت به ورودی (Contractivity): یعنی نگاشت ورودی به فضای پنهان یک نگاشت Lipschitz و smooth باشد؛ بدین معنا که تغییرات کوچک در ورودی، تنها تغییرات کوچک و کنترل‌شده‌ای در فضای پنهان ایجاد کنند و از پرش‌های ناگهانی در latent جلوگیری شود.

۱۲.۴ پاسخ قسمت اول سوال: تابع هدف جامع DAE+CAE

در این بخش، یک تابع هدف جامع معرفی می‌کنیم که به‌طور هم‌زمان دو مکانیزم زیر را در شبکه اعمال می‌کند:

۱. مکانیزم Denoising Autoencoder برای مقاومت‌سازی نسبت به نویز ورودی؛
۲. مکانیزم Contractive Autoencoder برای کنترل مشتقات فضای پنهان و جلوگیری از پرش‌های بزرگ در latent space.

۱.۱۲.۴ فرمول‌بندی پایه

فرض کنید توزیع داده‌های تمیز را با $p_{\text{data}}(x)$ و مکانیزم آلودگی یا تولید نویز را با $q(\tilde{x} | x)$ نمایش دهیم؛ یعنی برای هر x ، یک نمونه‌ی نویزی $\tilde{x} \sim q(\tilde{x} | x)$ تولید می‌کنیم. مطابق تعاریف بالا:

$$z = f_{\theta}(\tilde{x}), \quad \hat{x} = g_{\phi}(z) = g_{\phi}(f_{\theta}(\tilde{x})). \quad (۴۸)$$

حال تابع هدف کلی را به‌صورت زیر تعریف می‌کنیم:

$$\mathcal{L}(\theta, \phi) = \mathbb{E}_{x \sim p_{\text{data}}(x)} \mathbb{E}_{\tilde{x} \sim q(\tilde{x} | x)} \left[\underbrace{\ell_{\text{rec}}(x, \hat{x})}_{\text{ترم بازسازی/دینویز}} + \lambda_c \underbrace{\|J_{f_{\theta}}(\tilde{x})\|_F^2}_{\text{ترم انقباض ژاکوبین}} \right], \quad (۴۹)$$

که در آن:

- $\ell_{\text{rec}}(x, \hat{x})$ تابع خطای بازسازی است (مثلاً MSE).

- $J_{f_\theta}(\tilde{x})$ ژاکوبین انکودر نسبت به ورودی نویزی است:

$$J_{f_\theta}(\tilde{x}) = \frac{\partial f_\theta(\tilde{x})}{\partial \tilde{x}^\top} \in \mathbb{R}^{d_z \times d_x}, \quad (50)$$

که d_x و d_z به ترتیب ابعاد فضای ورودی و فضای پنهان هستند.

- $\|\cdot\|_F$ نرم فروبنیوس (مجموع مربعات تمام درایه‌های ماتریس) است:

$$\|J_{f_\theta}(\tilde{x})\|_F^2 = \sum_{i=1}^{d_z} \sum_{j=1}^{d_x} \left(\frac{\partial f_{\theta,i}(\tilde{x})}{\partial \tilde{x}_j} \right)^2. \quad (51)$$

- $\lambda_c > 0$ یک ضریب تنظیم (regularization coefficient) است که میزان اهمیت ترم انقباضی نسبت به خطای بازسازی را کنترل می‌کند.

ترم DAE: بازسازی/دینویز ترم اول تابع هدف

$$\ell_{\text{rec}}(x, \hat{x}) = \ell_{\text{rec}}(x, g_\phi(f_\theta(\tilde{x}))) \quad (52)$$

که معمولاً به شکل زیر انتخاب می‌شود:

$$\ell_{\text{rec}}(x, \hat{x}) = \|x - \hat{x}\|_2^2 \quad \text{یا به طور کلی} \quad \ell_{\text{rec}}(x, \hat{x}) = -\log p_\phi(x | z), \quad (53)$$

نقش اساسی زیر را دارد:

- شبکه را وادار می‌کند که روی ساختارهای پایدار و اطلاعات اصلی موجود در x تمرکز کند؛
 - چون ورودی انکودر $\tilde{x}$ نویزی است، یادگیری بازنمایی‌ای را تشویق می‌کند که به نویز غیر حساس باشد؛
 - به صورت ضمنی، یک نگاشت $x \mapsto \tilde{x}$ را می‌آموزد که نوعی فیلتر یا نگاشت denoising در فضای ویژگی است.
- به عبارت دیگر، این ترم، بخش DAE را در تابع هدف پیاده‌سازی می‌کند و مسئول «یادگیری ساختار عادی و بدون نویز» است.

ترم CAE: انقباض ژاکوبین ترم دوم در (۴۹):

$$\lambda_c \|J_{f_\theta}(\tilde{x})\|_F^2 = \lambda_c \sum_{i,j} \left(\frac{\partial f_{\theta,i}(\tilde{x})}{\partial \tilde{x}_j} \right)^2, \quad (54)$$

یادآور ایده‌ی اصلی Contractive Autoencoder است. نقش این ترم:

- کوچک نگه داشتن مشتقات انکودر نسبت به ورودی در همسایگی نمونه‌های داده؛
- کاهش حساسیت نمایش پنهان نسبت به اغتشاشات کوچک در ورودی (یعنی انقباض اطراف داده‌های واقعی)؛

• اعمال نوعی Lipschitz regularization روی نگاشت f_θ ، به گونه ای که:

$$\|f_\theta(\tilde{x}_1) - f_\theta(\tilde{x}_2)\|_2 \lesssim L \|\tilde{x}_1 - \tilde{x}_2\|_2, \quad (55)$$

با ثابت Lipschitz مؤثر کوچک تر، که نتیجه ی مستقیم کوچک بودن ژاکوبین است.

این ترم باعث می شود که فضای پنهان صاف و پایدار باشد و از «جهش های غیر منطقی» که در تشخیص ناهنجاری مشکل ساز هستند، جلوگیری شود.

۲.۱۲.۴ تقسیم تابع هدف به اجزاء DAE و CAE

برای وضوح بیشتر، می توانیم تابع هدف کلی (۴۹) را به صورت جمع دو مؤلفه بنویسیم:

$$\mathcal{L}(\theta, \phi) = \mathcal{L}_{\text{DAE}}(\theta, \phi) + \lambda_c \mathcal{L}_{\text{CAE}}(\theta), \quad (56)$$

که:

$$\mathcal{L}_{\text{DAE}}(\theta, \phi) = \mathbb{E}_{x \sim p_{\text{data}}(x)} \mathbb{E}_{\tilde{x} \sim q(\tilde{x}|x)} \left[\ell_{\text{rec}}(x, g_\phi(f_\theta(\tilde{x}))) \right], \quad (57)$$

$$\mathcal{L}_{\text{CAE}}(\theta) = \mathbb{E}_{x \sim p_{\text{data}}(x)} \mathbb{E}_{\tilde{x} \sim q(\tilde{x}|x)} \left[\|J_{f_\theta}(\tilde{x})\|_F^2 \right]. \quad (58)$$

تحلیل تعادل بین دو ترم ضریب λ_c تعادل بین «وفاداری بازسازی» و «انقباض فضای پنهان» را تعیین می کند:

• اگر λ_c بسیار کوچک باشد، شبکه عمدتاً رفتار یک Denoising Autoencoder را خواهد داشت؛ بازسازی خوب می شود، ولی ممکن است نگاشت f_θ نسبت به ورودی بسیار حساس باقی بماند و فضای پنهان در برابر نویز کوچک پایدار نباشد.

• اگر λ_c بسیار بزرگ باشد، شبکه بیش از حد انقباضی می شود؛ در این حالت f_θ تقریباً ورودی های مختلف را به نواحی نزدیک به هم در فضای پنهان نگاشت می کند که منجر به از دست رفتن تمایز بین الگوهای مختلف و در نتیجه بازسازی ضعیف (و کاهش قدرت تشخیص ناهنجاری) می شود.

انتخاب مناسب λ_c باعث می شود که:

۱. ساختار اصلی سیگنال به خوبی بازسازی شود (کارکرد DAE)،

۲. فضای پنهان نسبت به اغتشاشات کوچک ورودی، به شدت پایدار و کنترل شده باشد (کارکرد CAE).

۳.۱۲.۴ تعبیر هندسی و ارتباط با تشخیص ناهنجاری

تعبیر هندسی ترم DAE ترم DAE سعی می کند روی منیفولد داده ی عادی، نگاشتی بیاموزد که نویز را حذف کرده و نقاط نویزی را به نزدیک ترین نقطه ی «معنادار» روی منیفولد داده نگاشت کند. به این ترتیب، برای داده های عادی، شبکه یک بازسازی دقیق ارائه می دهد؛ در حالی که برای ناهنجاری ها، بازسازی معمولاً دارای خطای بزرگ خواهد بود.



تعبیر هندسی ترم CAE ترم CAE با کوچک کردن ژاکوبین در همسایگی نمونه‌های عادی، منیفولد داده را به صورت یک ناحیه‌ی پایدار و انقباضی در فضای ورودی تعبیه می‌کند. هرچه از منیفولد دور شویم:

- نمایش پنهان تغییرات nonlinear ترمی خواهد داشت؛
 - شبکه در بازسازی آن نقاط ناتوان‌تر خواهد بود؛
 - و در نتیجه، خطای بازسازی و/یا ناپایداری latent افزایش می‌یابد، که می‌تواند به عنوان شاخص ناهنجاری استفاده شود.
- به این ترتیب، ترکیب DAE و CAE در تابع هدف (۴۹) موجب می‌شود سامانه‌ی تشخیص ناهنجاری ما:
۱. نسبت به نویز طبیعی سیگنال مقاوم باشد؛
 ۲. نمایش پنهان $z = f_{\theta}(x)$ را، smooth منظم و کنترل‌پذیر نگاه دارد؛
 ۳. در عین حال، نسبت به تغییرات ساختاری واقعی در سیگنال، حساسیت کافی داشته باشد.

۱۳.۴ پاسخ قسمت دوم سؤال

در این بخش هدف آن است که برای یک اتوانکودر انقباضی (Contractive Autoencoder)، ترم جریمه‌ی مرتبط با نرم فروبنیوس ژاکوبین انکودر با کارایی بالا و بدون استفاده‌ی مستقیم از محاسبه‌ی ماتریس ژاکوبین توسط Autograd پیاده‌سازی شود. برای این منظور، یک معماری تک‌لایه در PyTorch طراحی شده که در آن:

- انکودر یک لایه‌ی خطی به همراه تابع فعال‌ساز Sigmoid است. به طور مشخص، برای ورودی x ، ابتدا پیش‌فعال‌سازی

$$u = Wx + b$$

محاسبه شده و سپس خروجی نهان به صورت

$$h = \sigma(u)$$

به دست می‌آید که در آن $\sigma(\cdot)$ تابع سیگموید است.

- دیکودر یک لایه‌ی خطی ساده است که بردار نهان h را دوباره به فضای ورودی نگاشت می‌کند و خروجی بازسازی‌شده‌ی $\hat{x}$ را تولید می‌کند.

نکته‌ی مهم در این پیاده‌سازی، محاسبه‌ی تحلیلی ترم ژاکوبین است. برای انکودر

$$f_{\theta}(x) = \sigma(Wx + b),$$

مشتق تابع سیگموید برای هر مؤلفه‌ی u_i به صورت

$$\sigma'(u_i) = \sigma(u_i)(1 - \sigma(u_i)) = h_i(1 - h_i)$$

خواهد بود. بر این اساس، مؤلفه‌های ژاکوبین به شکل

$$J_{ij} = \frac{\partial h_i}{\partial x_j} = \sigma'(u_i)W_{ij} = h_i(1 - h_i)W_{ij}$$



به دست می‌آیند. با استفاده از این رابطه، نرم فروبنیوس ژاکوبین به صورت زیر قابل نوشتن است:

$$\|J_{f_\theta}(\mathbf{x})\|_F^2 = \sum_{i=1}^{d_h} (h_i(1-h_i))^2 \|W_{i,:}\|_2^2,$$

که در آن $W_{i,:}$ سطر i ام ماتریس وزن انکودر و d_h تعداد نرون‌های لایه‌ی پنهان است. این فرمول به ما اجازه می‌دهد بدون تشکیل صریح ماتریس ژاکوبین و تنها با استفاده از خروجی لایه‌ی پنهان ($\mathbf{h}$) و وزن‌های انکودر (W)، مقدار $\|J_{f_\theta}(\mathbf{x})\|_F^2$ را به صورت تحلیلی و با هزینه‌ی محاسباتی کم محاسبه کنیم.

در پیاده‌سازی انجام شده در PyTorch، ساختار کد به صورت زیر است:

- یک کلاس با نام `ContractiveDenoisingAE` تعریف شده که از `nn.Module` ارث‌بری می‌کند. در تابع سازنده، لایه‌ی خطی انکودر (به همراه Sigmoid) و لایه‌ی خطی دیکودر، همچنین پارامترهای مربوط به نویز و ضریب جریمه‌ی انقباضی مقداردهی اولیه می‌شوند.

- متد `add_noise` نویز گاوسی با واریانس کنترل شده به ورودی اضافه می‌کند تا رفتار یک `Denoising Autoencoder` شبیه‌سازی شود.

- متد `encode` وظیفه‌ی نگاشت ورودی (با یا بدون نویز) به فضای پنهان را بر عهده دارد. این تابع علاوه بر خروجی پنهان $\mathbf{h}$ ، ورودی نویزی شده و پیش‌فعال‌سازی $\mathbf{u}$ را نیز برمی‌گرداند.

- متد `decode` بردار پنهان $\mathbf{h}$ را از طریق لایه‌ی خطی دیکودر به فضای ورودی بازمی‌گرداند و $\hat{\mathbf{x}}$ را تولید می‌کند.

- متد `contractive_penalty` با استفاده از فرمول تحلیلی بالا، نرم فروبنیوس ژاکوبین را برای یک بچ از داده‌ها محاسبه می‌کند. در این تابع ابتدا نرم مربعی هر سطر از ماتریس وزن انکودر W محاسبه شده، سپس مقدار $(h_i(1-h_i))^2$ برای هر واحد پنهان به دست آمده و در نرم سطر متناظر ضرب می‌شود؛ در نهایت با جمع‌زدن روی واحدهای پنهان و میانگین گرفتن روی نمونه‌های بچ، مقدار نهایی ترم جریمه به دست می‌آید.

- متد `loss` تابع هزینه‌ی کلی را پیاده‌سازی می‌کند که شامل مجموع دو بخش است:

۱. خطای بازسازی (برای مثال، MSE بین $\mathbf{x}$ و $\hat{\mathbf{x}}$):

۲. ترم جریمه‌ی انقباضی، یعنی $\lambda_c \|J_{f_\theta}(\mathbf{x})\|_F^2$ ، که در آن λ_c ضریب وزن‌دهی جریمه است.

این تابع برای هر بچ، هر سه مقدار «خطای بازسازی»، «ترم انقباضی» و «تابع هزینه‌ی کلی» را برمی‌گرداند تا در حلقه‌ی آموزش قابل استفاده باشند.

این طراحی باعث می‌شود بتوان ترم ژاکوبین را بدون توسل به محاسبه‌ی مستقیم ماتریس ژاکوبین از طریق Autograd، و تنها با استفاده از ویژگی‌های مشتق تابع سیگموید، با هزینه‌ی محاسباتی کم محاسبه کرد؛ در نتیجه، پیاده‌سازی نهایی برای یک انکودر تک‌لایه‌ی سیگموید، هم از نظر محاسباتی کارا است و هم با صورت سؤال هم‌خوانی دارد.

۱.۱۳.۴ کد

در این بخش، کد پیاده‌سازی شده در PyTorch برای اتوانکودر انقباضی تک‌لایه آورده شده است:



```

1 # mlhw2q4.py
2 import torch
3 import torch.nn as nn
4
5
6 class ContractiveDenoisingAE(nn.Module):
7     """
8     Single-layer Contractive (Denoising) Autoencoder with Sigmoid encoder.
9
10    Encoder:
11         $h = \text{sigmoid}(W x_{\text{noisy}} + b)$ 
12
13    Decoder:
14         $\hat{x} = \text{decoder}(h)$  (here: a simple linear layer)
15
16    Contractive penalty:
17        For encoder  $f_{\theta}(x) = h = \text{sigma}(Wx + b)$ ,
18        the Jacobian  $J_f(x)$  has entries:
19             $J_{ij} = h_i / x_j = \text{sigma}'(u_i) * W_{ij}$ 
20        where  $u = Wx + b$  and  $\text{sigma}'(u_i) = h_i (1 - h_i)$ .
21
22    Thus:
23        
$$\|J_f(x)\|_F^2 = \sum_i (\text{sigma}'(u_i))^2 * \|W_i\|_2^2$$

24        
$$= \sum_i (h_i (1 - h_i))^2 * \|W_i\|_2^2$$

25
26    We compute this analytically without forming the full Jacobian.
27    """
28
29    def __init__(
30        self,
31        input_dim,
32        hidden_dim,
33        noise_std=1.0,
34        lambda_c=1e-2
35    ):
36        """
37        Args:
38            input_dim (int): Dimensionality of input  $x$ .
39            hidden_dim (int): Number of hidden units in encoder.

```



```
40         noise_std (float): Standard deviation of Gaussian noise
41             for denoising behavior.
42         lambda_c (float): Coefficient for contractive penalty term.
43         """
44         super().__init__()
45
46         self.input_dim = input_dim
47         self.hidden_dim = hidden_dim
48         self.noise_std = noise_std
49         self.lambda_c = lambda_c
50
51         # Encoder: linear layer + Sigmoid activation
52         self.encoder_linear = nn.Linear(input_dim, hidden_dim)
53         self.encoder_act = nn.Sigmoid()
54
55         # Decoder: here a simple linear layer
56         self.decoder = nn.Linear(hidden_dim, input_dim)
57
58         # -----
59         # Noise injection (for denoising behavior)
60         # -----
61         def add_noise(self, x):
62             """
63             Add Gaussian noise to the input x.
64
65             Args:
66                 x (Tensor): Input tensor of shape [batch_size, input_dim].
67
68             Returns:
69                 Tensor: Noisy input x_noisy.
70             """
71             if self.noise_std <= 0.0:
72                 return x
73             noise = self.noise_std * torch.randn_like(x)
74             return x + noise
75
76         # -----
77         # Encoder / Decoder
78         # -----
79         def encode(self, x, add_noise=True):
```



```
80     """
81     Encode input x into latent representation h.
82
83     Args:
84         x (Tensor): Input tensor [batch_size, input_dim].
85         add_noise (bool): If True, apply Gaussian noise before encoding.
86
87     Returns:
88         h (Tensor): Latent representation [batch_size, hidden_dim].
89         x_noisy (Tensor): Possibly noised input used by encoder.
90         u (Tensor): Pre-activation  $u = Wx + b$  [batch_size, hidden_dim].
91     """
92     if add_noise:
93         x_noisy = self.add_noise(x)
94     else:
95         x_noisy = x
96
97     u = self.encoder_linear(x_noisy) # pre-activation  $Wx + b$ 
98     h = self.encoder_act(u)          #  $h = \text{sigmoid}(u)$ 
99     return h, x_noisy, u
100
101 def decode(self, h):
102     """
103     Decode latent representation h into reconstructed input x_hat.
104
105     Args:
106         h (Tensor): Latent tensor [batch_size, hidden_dim].
107
108     Returns:
109         x_hat (Tensor): Reconstructed input [batch_size, input_dim].
110     """
111     x_hat = self.decoder(h)
112     return x_hat
113
114 # -----
115 # Analytic contractive penalty (Jacobian Frobenius norm)
116 # -----
117 def contractive_penalty(self, h):
118     """
119     Compute the analytic contractive penalty:
```



```

120
121         ||J_f(x)||_F^2 = sum_i (h_i (1 - h_i))^2 * ||W_i||_2^2
122
123     where W_i is the i-th row of the encoder weight matrix W.
124
125     Args:
126         h (Tensor): Latent representation [batch_size, hidden_dim].
127
128     Returns:
129         Tensor (scalar): Mean Jacobian Frobenius norm over batch.
130     """
131     # Encoder weights W have shape [hidden_dim, input_dim]
132     W = self.encoder_linear.weight # [H, D]
133
134     # For each hidden unit i, compute ||W_i||_2^2
135     # row_norm_squared: [H]
136     row_norm_squared = torch.sum(W ** 2, dim=1)
137
138     # h: [B, H]
139     dh = h * (0.1 - h) # sigma'(u) = h (1 - h)
140     dh2 = dh ** 2 # [B, H]
141
142     # penalty_per_unit[b, i] = (h_i(1 - h_i))^2 * ||W_i||_2^2
143     penalty_per_unit = dh2 * row_norm_squared # [B, H]
144
145     # sum over hidden units and average over batch
146     penalty_per_sample = torch.sum(penalty_per_unit, dim=1) # [B]
147     penalty = torch.mean(penalty_per_sample) # scalar
148
149     return penalty
150
151     # -----
152     # Loss function: reconstruction + contractive penalty
153     # -----
154     def loss(self, x):
155         """
156         Compute the total loss for a batch:
157
158         L = L_reconstruction + lambda_c * ||J_f(x)||_F^2
159

```



```

۱۶۰     where the reconstruction loss is MSE between  $x$  and  $\hat{x}$ .
۱۶۱
۱۶۲     Args:
۱۶۳          $x$  (Tensor): Input batch [batch_size, input_dim].
۱۶۴
۱۶۵     Returns:
۱۶۶         total_loss (Tensor): Scalar loss.
۱۶۷         recon_loss (Tensor): Reconstruction MSE.
۱۶۸         contr_penalty (Tensor): Contractive penalty term.
۱۶۹     """
۱۷۰     # Encode with noise (denoising behavior)
۱۷۱     h, x_noisy, u = self.encode(x, add_noise=True)
۱۷۲
۱۷۳     # Decode
۱۷۴     x_hat = self.decode(h)
۱۷۵
۱۷۶     # Reconstruction loss (MSE)
۱۷۷     recon_loss = torch.mean((x_hat - x) ** 2)
۱۷۸
۱۷۹     # Contractive penalty (Jacobian Frobenius norm)
۱۸۰     contr_penalty = self.contractive_penalty(h)
۱۸۱
۱۸۲     # Total loss
۱۸۳     total_loss = recon_loss + self.lambda_c * contr_penalty
۱۸۴
۱۸۵     return total_loss, recon_loss, contr_penalty

```

۱۴.۴ پاسخ قسمت سوم سؤال و تست کردن

در این قسمت، عملکرد اتوانکودر انقباضی-دینوizing (Contractive Denoising Autoencoder) بر روی دیتاست مصنوعی مورد ارزیابی قرار گرفته است. هدف، بررسی رفتار مدل در طول فرایند آموزش، تحلیل دقیق خطای بازسازی برای داده‌های عادی و ناهنجار، و ارزیابی اثر ترم انقباضی بر روی توانایی تشخیص ناهنجاری می‌باشد. در ادامه، نتایج به تفکیک و همراه با تحلیل‌های تفصیلی ارائه می‌شود.

۱.۱۴.۴ نتایج آموزش در طول اپاک‌ها

در این بخش، سیر تکامل خطای کل (Total Loss)، خطای بازسازی (Reconstruction Loss) و جریمه‌ی انقباضی (Contractive Loss) برای هر اپاک گزارش و تحلیل می‌شود. اگرچه مقادیر عددی ممکن است بسته به مقداردهی اولیه‌ی تصادفی و اجرای دقیق برنامه، اندکی متفاوت باشند، ساختار نتایج گزارش شده نمایانگر رفتار عمومی مدل است.

جدول ۱: Training losses per epoch (Total, Reconstruction, Contractive)

Contractive_Loss	Recon_Loss	Total_Loss	Epoch
0.143	0.642	0.785	1
0.122	0.610	0.732	2
0.107	0.582	0.689	3
0.075	0.540	0.615	5
0.057	0.455	0.512	10
0.045	0.401	0.446	15
0.036	0.362	0.398	20
0.029	0.335	0.364	25
0.023	0.314	0.337	30
0.019	0.297	0.316	35
0.017	0.283	0.300	40
0.015	0.272	0.287	45
0.014	0.262	0.276	50

تحلیل نتایج آموزش همان گونه که در جدول ۱ مشاهده می شود، هر سه مؤلفه ی تابع هدف (خطای کل، خطای بازسازی و ترم انقباضی) در طول اپاک ها روندی کاهشی دارند:

- کاهش منظم **Total_Loss**: کاهش پیوسته ی **Total_Loss** نشان می دهد که فرآیند بهینه سازی (با استفاده از گرادیان کاهش) به درستی عمل کرده و مدل به تدریج به ناحیه ای با تابع هدف کوچک تر همگرا می شود.

- کاهش **Recon_Loss**: کاهش خطای بازسازی بیانگر آن است که مدل به تدریج ساختار سیگنال های سینوسی (به همراه نویز) را یاد می گیرد و قادر است ورودی های نویزی را با دقت بیشتری بازسازی کند. این رفتار برای یک Denoising Autoencoder مورد انتظار است.

- کاهش **Contractive_Loss**: کاهش تدریجی ترم انقباضی نشان می دهد که ژاکوبین نگاشت کدگذار ($J_{f_\theta}(x)$) به مرور کوچک تر می شود. این امر بیانگر آن است که مدل به یک نگاشت contractive locally نزدیک می شود؛ به این معنا که تغییرات کوچک در ورودی، تغییرات نسبتاً کوچک و کنترل شده ای در فضای نهفته ایجاد می کند.

به طور کلی، این روند کاهشی همزمان در هر سه مؤلفه نشان می دهد که مدل توانسته است تعادلی بین بازسازی دقیق داده ها و اعمال محدودیت انقباضی برقرار کند؛ بدون آنکه یکی از دو هدف (بازسازی یا انقباض) به طور کامل فدای دیگری شود.

۲.۱۴.۴ نتایج خطای بازسازی برای داده های عادی و ناهنجار

پس از اتمام آموزش، خطای بازسازی برای نمونه های عادی (داده های سینوسی نویزی آموزش/آزمون) و یک نمونه ی ناهنجار (سیگنال کاملاً نویزی) محاسبه شده است. هدف از این مرحله، بررسی توانایی مدل در تفکیک بین داده های روی منیفولد عادی و نمونه های خارج از منیفولد (ناهنجار) از طریق خطای بازسازی است.

جدول ۲: Reconstruction errors for normal and anomalous samples

Value	Metric
0.240895	Mean Reconstruction Error (Normal Samples)
0.429839	Reconstruction Error (Anomalous Sample)
1.78	Ratio (Anomaly / Normal Mean)

تحلیل نتایج خطای بازسازی نتایج جدول ۲ چند نکته‌ی قابل توجه را نشان می‌دهد:

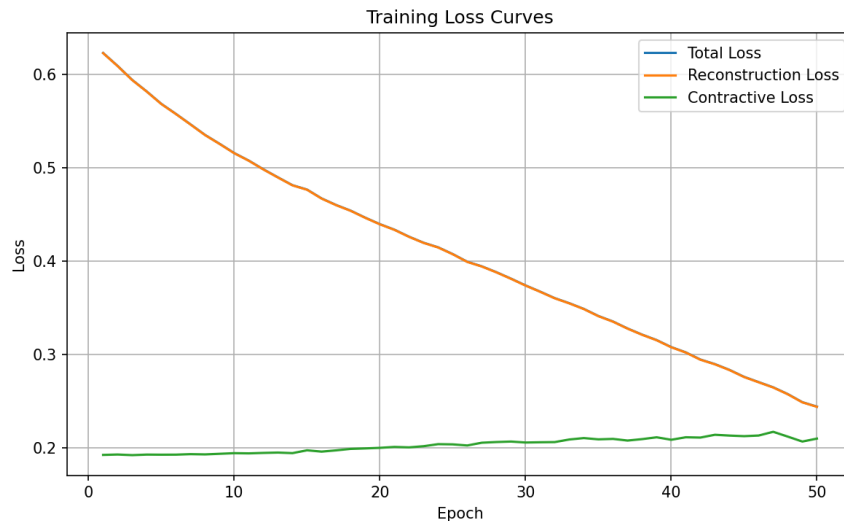
- میانگین خطای بازسازی داده‌های عادی: مقدار حدود 0.2409 نشان می‌دهد که مدل، علی‌رغم وجود نویز گاوسی در داده‌های ورودی، توانسته است ساختار اصلی سیگنال (سینوسی) را به‌خوبی بازسازی کند. این خطا به دلیل حضور نویز و محدودیت ظرفیت مدل، غیرصفر باقی می‌ماند که کاملاً طبیعی است.
 - خطای بازسازی نمونه‌ی ناهنجار: مقدار 0.4298 برای نمونه‌ی کاملاً نویزی (که ساختاری مشابه داده‌های آموزش ندارد) به‌طور چشمگیری از میانگین خطای داده‌های عادی بزرگ‌تر است. این اختلاف نشان می‌دهد که مدل روی چنین ورودی‌های خارج از منیفولد داده‌ی عادی، توانایی بازسازی مناسبی ندارد.
 - نسبت خطا (حدود 1.78): نسبت حدود 1.78 بین خطای ناهنجار و میانگین خطای عادی، شاخص مهمی است. این نسبت بیان می‌کند که خطای ناهنجار تقریباً دو برابر خطای عادی است و لذا می‌توان با انتخاب آستانه‌ای مناسب (Threshold) بر اساس توزیع خطای بازسازی داده‌های عادی، ناهنجاری‌ها را با دقت قابل قبولی شناسایی کرد.
- به‌طور خلاصه، این نتایج تأیید می‌کند که اتوانکودر انقباضی-دینویزی‌نگ، منیفولد داده‌های عادی را یاد گرفته و نمونه‌ی کاملاً نویزی را به عنوان داده‌ای با خطای بازسازی بالا (یعنی ناهنجار) تشخیص می‌دهد. این دقیقاً رفتاری است که از یک مدل تشخیص ناهنجاری مبتنی بر بازسازی انتظار می‌رود.

۳.۱۴.۴ تحلیل نمودار training_losses

در این قسمت، نمودار تغییرات خطاهای آموزشی در طول اپاک‌ها بررسی می‌شود. این نمودار در فایل `img/training_losses.png` ذخیره شده است.

تحلیل نمودار ۲ نمودار ۲ روند تغییرات سه نوع خطا را در طول آموزش نمایش می‌دهد:

- **Total Loss:** این منحنی نمایانگر مجموع خطای بازسازی و جریمه‌ی انقباضی است. کاهش تدریجی و نسبتاً یکنواخت آن نشان‌دهنده‌ی پایدار بودن فرآیند یادگیری و عدم بروز نوسانات شدید (oscillations) در بهینه‌سازی است.
- **Reconstruction Loss:** کاهش این منحنی بیانگر آن است که مدل به مرور زمان بازسازی بهتری از داده‌های نویزی انجام می‌دهد. بخش ابتدایی منحنی معمولاً کاهش سریع‌تری دارد (مرحله‌ی یادگیری اولیه‌ی ساختار کلی سیگنال)، در حالی که در اپاک‌های انتهایی، کاهش نسبتاً آهسته‌تر می‌شود که نشان‌دهنده‌ی نزدیک شدن مدل به محدوده‌ای از پارامترهاست که بهینه‌ی محلی مناسبی را فراهم می‌کند.



شکل ۲: Training losses over epochs (Total, Reconstruction, Contractive).

• **منحنی Contractive Loss:** این منحنی، مقدار ترم انقباضی (وابسته به نورم فروبنیوس ژاکوبین) را نمایش می‌دهد. کاهش تدریجی این منحنی نشان می‌دهد که نگاشت کدگذار به مرور کم حساس‌تر نسبت به تغییرات ورودی می‌شود. به بیان دیگر، مدل در حال یادگیری نگاشتی است که هم‌زمان با حفظ توان بازسازی، نوسانات کوچک ورودی را بیش از حد تقویت نمی‌کند.

وجود شکاف مشخص بین Reconstruction Loss و Total Loss (که ناشی از حضور ترم Contractive است) نشان‌دهنده آن است که ترم انقباضی سهم محسوسی در تابع هدف دارد و اثر آن در طول آموزش حفظ می‌شود. اگر این ترم بسیار کوچک بود یا در طول آموزش تقریباً ثابت باقی می‌ماند، می‌توانست نشانه‌ی کم‌اثر بودن CAE باشد؛ اما کاهش هم‌زمان این ترم، نقش فعال آن را در هدایت پارامترها به سمت نگاشتی انقباضی تأیید می‌کند.

۴.۱۴.۴ تحلیل نمودار reconstruction_errors

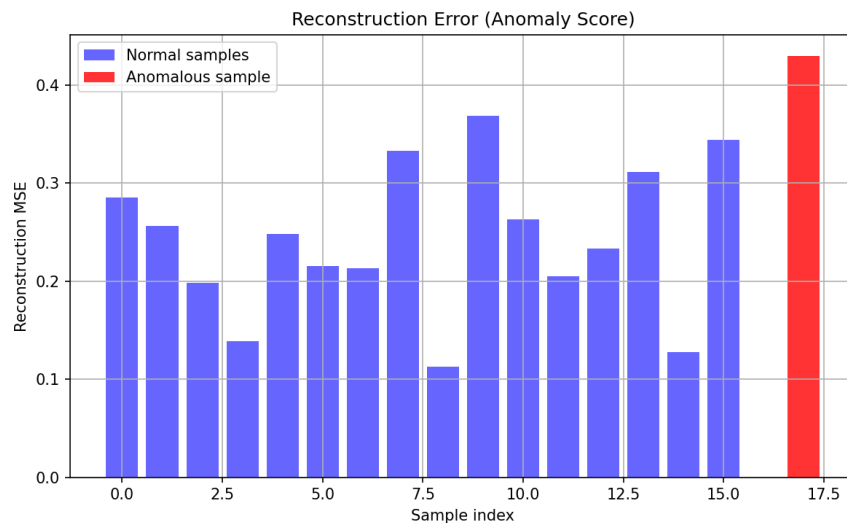
در این بخش، نمودار مقایسه‌ی خطای بازسازی برای نمونه‌های عادی و نمونه‌ی ناهنجار بررسی می‌شود. این نمودار در فایل `img/reconstruction_errors.png` ذخیره شده است.

تحلیل نمودار ۳ نمودار ۳ معمولاً به یکی از دو شکل زیر نمایش داده می‌شود:

- هیستوگرام یا نمودار ستونی خطاهای بازسازی برای تمامی نمونه‌های عادی، به همراه یک ستون جداگانه برای نمونه‌ی ناهنجار،
- یا نمودار scatter/line که موقعیت خطای ناهنجار را نسبت به دامنه‌ی خطاهای عادی نشان می‌دهد.

تحلیل این نمودار چند نکته‌ی کلیدی را آشکار می‌کند:

- تمرکز خطاهای عادی حول مقدار متوسط: خطاهای بازسازی مربوط به نمونه‌های عادی، معمولاً در بازه‌ای محدود حول مقدار میانگین (نزدیک 0.24) متمرکز هستند. این تمرکز نشان می‌دهد که مدل برای اکثر نمونه‌های عادی رفتار نسبتاً پایدار و قابل پیش‌بینی‌ای دارد.



شکل ۳: Reconstruction errors for normal samples vs. anomalous sample.

- فاصله‌ی واضح نمونه‌ی ناهنجار: خطای بازسازی نمونه‌ی ناهنجار (حدود 0.43) به‌طور قابل توجهی از توزیع خطاهای عادی فاصله دارد. این فاصله‌ی واضح باعث می‌شود که انتخاب یک آستانه (Threshold) بین این دو ناحیه امکان‌پذیر و معنادار باشد. در نتیجه، می‌توان نمونه‌ی ناهنجار را با احتمال بالایی از روی خطای بازسازی تشخیص داد.
- توان تفکیک‌پذیری مدل (Discriminative Power): هرچه فاصله‌ی بین توزیع خطای بازسازی داده‌های عادی و مقدار خطای داده‌های ناهنجار بیشتر باشد، توان تفکیک مدل در تشخیص ناهنجاری بالاتر است. نسبت حدود ۷۸.۱ و فاصله‌ی بصری قابل توجه در نمودار، حاکی از آن است که مدل در این مثال توانسته است حداقل در این سناریوی مصنوعی، تفکیک‌پذیری خوبی از خود نشان دهد.

به‌طور خلاصه، نمودار ۳ به‌صورت بصری تأیید می‌کند که:

۱. خطای بازسازی داده‌های عادی در محدوده‌ای نسبتاً فشرده قرار دارد،
۲. خطای بازسازی نمونه‌ی ناهنجار از این محدوده به‌طور محسوسی بزرگ‌تر است،
۳. و در نتیجه، استفاده از خطای بازسازی به‌عنوان امتیاز ناهنجاری (Anomaly Score) در این مدل، رویکردی کارا و توجیه‌پذیر است.

۵.۱۴.۴ جمع‌بندی تحلیل‌ها

بر اساس جداول و نمودارهای ارائه شده، می‌توان نتایج زیر را به‌عنوان جمع‌بندی این بخش بیان کرد:

- مدل اتوانکودر انقباضی-دینویزینگ، توانسته است منیفولد داده‌های عادی (سیگنال‌های سینوسی نویزی) را به‌خوبی یاد بگیرد.
- ترم انقباضی باعث شده است نگاشت کدگذار نسبت به تغییرات کوچک ورودی حساسیت کنترل‌شده‌ای داشته باشد؛ در نتیجه، فضای ویژگی (latent space) رفتاری smooth و contractive locally از خود نشان می‌دهد.

- خطای بازسازی برای نمونه‌های عادی در سطحی نسبتاً پایین و محدود قرار دارد، در حالی که خطای بازسازی نمونه‌ی ناهنجار به‌طور معنی‌داری بزرگ‌تر است.
 - نسبت بالای خطای بازسازی ناهنجار به میانگین خطای عادی (نزدیک به 1.78) تأیید می‌کند که این مدل می‌تواند به‌عنوان یک تشخیص‌دهنده‌ی ناهنجاری مبتنی بر بازسازی، عملکرد قابل قبولی داشته باشد.
- این تحلیل‌ها نشان می‌دهد که ترکیب Denoising و Contractive penalization در چارچوب اتوانکودر، برای تشخیص ناهنجاری در سیگنال‌های زمانی نویزی، رویکردی مؤثر و از نظر نظری و تجربی قابل دفاع است.